so that $y^{(1)}$ and $y^{(2)}$ agree to the first term, viz., $(1/3)x^3$. To find the range of values of x so that the series with the term $(1/3)x^3$ alone will give the result correct to three decimal places, we put

$$\frac{1}{81}x^9 \le 0.0005$$

which yields

$$x \le 0.7$$

Hence

$$y(0.25) = \frac{1}{3}(0.25)^3 = 0.005$$

$$y(0.5) = \frac{1}{3}(0.5)^3 = 0.042$$

$$y(1.0) = \frac{1}{3} - \frac{1}{81} = 0.321$$

8.4 EULER'S METHOD

We have so far discussed the methods which yield the solution of a differential equation in the form of a power series. We will now describe the methods which give the solution in the form of a set of tabulated values.

Suppose that we wish to solve the Eqs. (8.1) for values of y at $x = x_r = x_0 + rh (r = 1, 2, \ldots)$. Integrating Eq. (8.1), we obtain

$$y_1 = y_0 + \int_{x_0}^{x_1} f(x, y)\,dx. \tag{8.6}$$

Assuming that $f(x, y) = f(x_0, y_0)$ in $x_0 \le x \le x_1$, this gives Euler's formula

$$y_1 \approx y_0 + hf(x_0, y_0). \tag{8.7a}$$

Similarly for the range $x_1 \le x \le x_2$, we have

$$y_2 = y_1 + \int_{x_1}^{x_2} f(x, y)\,dx.$$

Substituting $f(x_1, y_1)$ for $f(x, y)$ in $x_1 \le x \le x_2$ we obtain

$$y_2 \approx y_1 + hf(x_1, y_1). \tag{8.7b}$$

Proceeding in this way, we obtain the general formula

$$y_{n+1} = y_n + hf(x_n, y_n), \quad n = 0, 1, 2, \ldots \tag{8.8}$$

The process is very slow and to obtain reasonable accuracy with Euler's method, we need to take a smaller value for h. Because of this restriction

on h, the method is unsuitable for practical use and a modification of it, known as the *modified Euler method*, which gives more accurate results, will be described in Section 8.4.2.

Example 8.5 To illustrate Euler's method, we consider the differential equation $y' = -y$ with the condition $y(0) = 1$.

Successive application of Eq. (8.8) with $h = 0.01$ gives

$$y(0.01) = 1 + 0.1(-1) = 0.99$$

$$y(0.02) = 0.99 + 0.01(-0.99) = 0.9801$$

$$y(0.03) = 0.9801 + 0.01(-0.9801) = 0.9703$$

$$y(0.04) = 0.9703 + 0.01(-0.9703) = 0.9606.$$

The exact solution is $y = e^{-x}$ and from this the value at $x = 0.04$ is 0.9608.

8.4.1 Error Estimates for the Euler Method

Let the true solution of the differential equation at $x = x_n$ be $y(x_n)$ and also let the approximate solution be y_n. Now, expanding $y(x_{n+1})$ by Taylor's series, we get

$$\begin{aligned} y(x_{n+1}) &= y(x_n) + hy'(x_n) + \frac{h^2}{2} y''(x_n) + \cdots \\ &= y(x_n) + hy'(x_n) + \frac{h^2}{2} y''(\tau_n), \qquad \text{where } x_n \le \tau_n \le x_{n+1}. \end{aligned} \tag{8.9}$$

We usually encounter two types of errors in the solution of differential equations. These are (i) local errors, and (ii) rounding errors. The local error is the result of replacing the given differential equation by means of the equation

$$y_{n+1} = y_n + hy'_n.$$

This error is given by

$$L_{n+1} = -\frac{1}{2} h^2 y''(\tau_n) \tag{8.10}$$

The total error is then defined by

$$e_n = y_n - y(x_n) \tag{8.11}$$

Since y_0 is exact, it follows that $e_0 = 0$.

Neglecting the rounding error, we write the total solution error as

$$\begin{aligned} e_{n+1} &= y_{n+1} - y(x_{n+1}) \\ &= y_n + hy'_n - [y(x_n) + hy'(x_n) - L_{n+1}] \\ &= e_n + h[f(x_n, y_n) - y'(x_n)] + L_{n+1}. \end{aligned}$$

$$\Rightarrow \qquad e_{n+1} = e_n + h\left[f(x_n, y_n) - f(x_n, y(x_n))\right] + L_{n+1}.$$

By mean value theorem, we write

$$f(x_n, y_n) - f(x_n, y(x_n)) = \left[y_n - y(x_n)\right]\frac{\partial f}{\partial y}(x_n, \xi_n), \quad y(x_n) \le \xi_n \le y_n.$$

Hence, we have

$$e_{n+1} = e_n\left[1 + hf_y(x_n, \xi_n)\right] + L_{n+1} \tag{8.12}$$

Since $e_0 = 0$, we obtain successively:

$$e_1 = L_1; \quad e_2 = \left[1 + hf_y(x_1, \xi_1)\right]L_1 + L_2;$$

$$e_3 = \left[1 + hf_y\,(x_2, \xi_2)\right]\left[1 + hf_y(x_1, \xi_1)\right](L_1 + L_2) + L_3; \text{ etc.}$$

See the book by Isaacson and Keller [1966] for more details.

Example 8.6 We consider, again, the differential equation $y' = -y$ with the condition $y(0) = 1,$ which we have solved by Euler's method in Example 8.5.

Choosing $h = 0.01,$ we have

$$1 + hf_y(x_n, \xi_n) = 1 + 0.01(-1) = 0.99.$$

and

$$L_{n+1} = -\frac{1}{2}h^2 y''(\rho_n) = -0.00005\, y(\rho_n).$$

In this problem, $y(\rho_n) \le y(x_n),$ since y' is negative. Hence we successively obtain

$$|L_1| \le 0.00005 = 5\times 10^{-5},$$

$$|L_2| \le (0.00005)\,(0.99) < 5\times 10^{-5},$$

$$|L_3| \le (0.00005)\,(0.9801) < 5\times 10^{-5},$$

and so on. For computing the total solution error, we need an estimate of the rounding error. If we neglect the rounding error, i.e., if we set

$$R_{n+1} = 0,$$

then using the above bounds, we obtain from Eq. (8.12) the estimates

$$e_0 = 0,$$

$$|e_1| \le 5\times 10^{-5}$$

$$|e_2| \le 0.99e_1 + 5\times 10^{-5} < 10^{-4}$$

$$|e_3| \le 0.99e_2 + 5\times 10^{-5} < 10^{-4} + 5\times 10^{-5}$$

$$|e_4| \le 0.99e_3 + 5\times 10^{-5} < 10^{-4} + 10^{-4} = 2\times 10^{-4} = 0.0002$$

$$\vdots$$

It can be verified that the estimate for e_4 agrees with the actual error in the value of $y(0.04)$ obtained in Example 8.5.

8.4.2 Modified Euler's Method

Instead of approximating $f(x, y)$ by $f(x_0, y_0)$ in Eq. (8.6), we now approximate the integral given in Eq. (8.6) by means of trapezoidal rule to obtain

$$y_1 = y_0 + \frac{h}{2}[f(x_0, y_0) + f(x_1, y_1)] \tag{8.13}$$

We thus obtain the iteration formula

$$y_1^{(n+1)} = y_0 + \frac{h}{2}[f(x_0, y_0) + f(x_1, y_1^{(n)})], \quad n = 0, 1, 2, \ldots \tag{8.14}$$

where $y_1^{(n)}$ is the nth approximation to y_1. The iteration formula (8.14) can be started by choosing $y_1^{(0)}$ from Euler's formula:

$$y_1^{(0)} = y_0 + hf(x_0, y_0).$$

Example 8.7 Determine the value of y when $x = 0.1$ given that

$$y(0) = 1 \quad \text{and} \quad y' = x^2 + y$$

We take $h = 0.05$. With $x_0 = 0$ and $y_0 = 1.0$, we have $f(x_0, y_0) = 1.0$. Hence Euler's formula gives

$$y_1^{(0)} = 1 + 0.05(1) = 1.05$$

Further, $x_1 = 0.05$ and $f(x_1, y_1^{(0)}) = 1.0525$. The average of $f(x_0, y_0)$ and $f(x_1, y_1^{(0)})$ is 1.0262. The value of $y_1^{(1)}$ can therefore be computed by using Eq. (8.14) and we obtain

$$y_1^{(1)} = 1.0513.$$

Repeating the procedure, we obtain $y_1^{(2)} = 1.0513$. Hence we take $y_1 = 1.0513$, which is correct to four decimal places.

Next, with $x_1 = 0.05$, $y_1 = 1.0513$ and $h = 0.05$, we continue the procedure to obtain y_2, i.e., the value of y when $x = 0.1$. The results are

$$y_2^{(0)} = 1.1040, \quad y_2^{(1)} = 1.1055, \quad y_2^{(2)} = 1.1055.$$

Hence we conclude that the value of y when $x = 0.1$ is 1.1055.

8.5 RUNGE–KUTTA METHODS

As already mentioned, Euler's method is less efficient in practical problems since it requires h to be small for obtaining reasonable accuracy. The

Runge–Kutta methods are designed to give greater accuracy and they possess the advantage of requiring only the function values at some selected points on the subinterval.

If we substitute $y_1 = y_0 + hf(x_0, y_0)$ on the right side of Eq. (8.13), we obtain

$$y_1 = y_0 + \frac{h}{2}[f_0 + f(x_0 + h, y_0 + hf_0)],$$

where $f_0 = f(x_0, y_0)$. If we now set

$$k_1 = hf_0 \quad \text{and} \quad k_2 = hf(x_0 + h,\ y_0 + k_1)$$

then the above equation becomes

$$y_1 = y_0 + \frac{1}{2}(k_1 + k_2), \tag{8.15}$$

which is the *second-order Runge–Kutta* formula. The error in this formula can be shown to be of order h^3 by expanding both sides by Taylor's series. Thus, the left side gives

$$y_0 + hy_0' + \frac{h^2}{2}y_0'' + \frac{h^3}{6}y_0''' + \cdots$$

and on the right side

$$k_2 = hf\ (x_0 + h, y_0 + hf_0) = h\left[f_0 + h\frac{\partial f}{\partial x_0} + hf_0\frac{\partial f}{\partial y_0} + \mathrm{O}(h^2)\right].$$

Since

$$\frac{df\,(x, y)}{dx} = \frac{\partial f}{\partial x} + f\frac{\partial f}{\partial y},$$

we obtain

$$k_2 = h[f_0 + hf_0' + \mathrm{O}(h^2)] = hf_0 + h^2 f_0' + \mathrm{O}(h^3),$$

so that the right side of Eq. (8.15) gives

$$y_0 + \frac{1}{2}[hf_0 + hf_0 + h^2 f_0' + \mathrm{O}(h^3)] = y_0 + hf_0 + \frac{1}{2}h^2 f_0' + \mathrm{O}(h^3)$$

$$= y_0 + hy_0' + \frac{h^2}{2}y_0'' + \mathrm{O}(h^3).$$

It therefore follows that the Taylor series expansions of both sides of Eq. (8.15) agree up to terms of order h^2, which means that the error in this formula is of order h^3.

More generally, if we set

$$y_1 = y_0 + W_1k_1 + W_2k_2 \tag{8.16a}$$

where

$$\left.\begin{aligned} k_1 &= hf_0 \\ k_2 &= hf\ (x_0 + \alpha_0 h,\ y_0 + \beta_0 k_1) \end{aligned}\right\} \tag{8.16b}$$

then the Taylor series expansions of both sides of the last equation in (8.16a) gives the identity

$$y_0 + hf_0 + \frac{h^2}{2}\left(\frac{\partial f}{\partial x} + f_0\frac{\partial f}{\partial y}\right) + \mathrm{O}(h^3) = y_0 + (W_1 + W_2)hf_0 + W_2h^2\left(\alpha_0\frac{\partial f}{\partial x} + \beta_0 f_0\frac{\partial f}{\partial y}\right) + \mathrm{O}(h^3).$$

Equating the coefficients of $f(x, y)$ and its derivatives on both sides, we obtain the relations

$$W_1 + W_2 = 1, \quad W_2\alpha_0 = \frac{1}{2}, \quad W_2\beta_0 = \frac{1}{2}. \tag{8.17}$$

Clearly, $\alpha_0 = \beta_0$ and if α_0 is assigned any value arbitrarily, then the remaining parameters can be determined uniquely. If we set, for example, $\alpha_0 = \beta_0 = 1$, then we immediately obtain $W_1 = W_2 = 1/2$, which gives formula (8.15).

It follows, therefore, that there are several second-order Runge–Kutta formulae and that formulae (8.16) and (8.17) constitute just one of several such formulae.

Higher-order Runge–Kutta formulae exist, of which we mention only the *fourth-order formula* defined by

$$y_1 = y_0 + W_1k_1 + W_2k_2 + W_3k_3 + W_4k_4 \tag{8.18a}$$

where

$$\left.\begin{aligned} k_1 &= hf(x_0, y_0) \\ k_2 &= hf(x_0 + \alpha_0 h, y_0 + \beta_0 k_1) \\ k_3 &= hf(x_0 + \alpha_1 h, y_0 + \beta_1 k_1 + \nu_1 k_2) \\ k_4 &= hf(x_0 + \alpha_2 h, y_0 + \beta_2 k_1 + \nu_2 k_2 + \delta_1 k_3), \end{aligned}\right\} \tag{8.18b}$$

where the parameters have to be determined by expanding both sides of the first equation of (8.18a) by Taylor's series and securing agreement of terms up to and including those containing h^4. The choice of the parameters is, again, arbitrary and we have therefore several fourth-order Runge–Kutta formulae. If, for example, we set

$$\left.\begin{aligned} &\alpha_0 = \beta_0 = \frac{1}{2}, && \alpha_1 = \frac{1}{2}, && \alpha_2 = 1, \\ &\beta_1 = \frac{1}{2}(\sqrt{2} - 1), && \beta_2 = 0 \\ &\nu_1 = 1 - \frac{1}{\sqrt{2}}, && \nu_2 = -\frac{1}{\sqrt{2}}, && \delta_1 = 1 + \frac{1}{\sqrt{2}}, \\ &W_1 = W_4 = \frac{1}{6}, && W_2 = \frac{1}{3}\left(1 - \frac{1}{\sqrt{2}}\right), && W_3 = \frac{1}{3}\left(1 + \frac{1}{\sqrt{2}}\right), \end{aligned}\right\} \tag{8.19}$$

we obtain the method of Gill, whereas the choice

$$\left.\begin{aligned} &\alpha_0 = \alpha_1 = \frac{1}{2}, && \beta_0 = \nu_1 = \frac{1}{2} \\ &\beta_1 = \beta_2 = \nu_2 = 0, && \alpha_2 = \delta_1 = 1 \\ &W_1 = W_4 = \frac{1}{6}, && W_2 = W_3 = \frac{2}{6} \end{aligned}\right\} \tag{8.20}$$

leads to the fourth-order Runge–Kutta formula, the most commonly used one in practice:

$$y_1 = y_0 + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{8.21a}$$

where

$$\left.\begin{aligned} k_1 &= hf(x_0, y_0) \\ k_2 &= hf\left(x_0 + \frac{1}{2}h, y_0 + \frac{1}{2}k_1\right) \\ k_3 &= hf\left(x_0 + \frac{1}{2}h, y_0 + \frac{1}{2}k_2\right) \\ k_4 &= hf(x_0 + h, y_0 + k_3) \end{aligned}\right\} \tag{8.21b}$$

in which the error is of order h^5. Complete derivation of the formula is exceedingly complicated, and the interested reader is referred to the book by Levy and Baggot. We illustrate here the use of the fourth-order formula by means of examples.

Example 8.8 Given $dy/dx = y - x$ where $y(0) = 2$, find $y(0.1)$ and $y(0.2)$ correct to four decimal places.

(i) *Runge–Kutta second-order formula:* With $h = 0.1$, we find $k_1 = 0.2$ and $k_2 = 0.21$. Hence

$$y_1 = y(0.1) = 2 + \frac{1}{2}(0.41) = 2.2050.$$

To determine $y_2 = y(0.2)$, we note that $x_0 = 0.1$ and $y_0 = 2.2050$. Hence, $k_1 = 0.1(2.105) = 0.2105$ and $k_2 = 0.1(2.4155 - 0.2) = 0.22155$.

It follows that

$$y_2 = 2.2050 + \frac{1}{2}(0.2105 + 0.22155) = 2.4210.$$

Proceeding in a similar way, we obtain

$$y_3 = y(0.3) = 2.6492 \quad \text{and} \quad y_4 = y(0.4) = 2.8909$$

We next choose $h = 0.2$ and compute $y(0.2)$ and $y(0.4)$ directly. With $h = 0.2$. $x_0 = 0$ and $y_0 = 2$, we obtain $k_1 = 0.4$ and $k_2 = 0.44$ and hence $y(0.2) = 2.4200$. Similarly, we obtain $y(0.4) = 2.8880$.

From the analytical solution $y = x + 1 + e^x$, the exact values of $y(0.2)$ and $y(0.4)$ are respectively 2.4214 and 2.8918. To study the order of convergence of this method, we tabulate the values as follows:

x	*Computed y*	*Exact y*	*Difference*	*Ratio*
0.2	$h = 0.1$: 2.4210	2.4214	0.0004	3.5
	$h = 0.2$: 2.4200		0.0014	
0.4	$h = 0.1$: 2.8909	2.8918	0.0009	4.2
	$h = 0.2$: 2.8880		0.0038	

It follows that the method has an h^2-order of convergence.

(ii) *Runge–Kutta fourth-order formula:* To determine $y(0.1)$, we have $x_0 = 0$, $y_0 = 2$ and $h = 0.1$. We then obtain

$$k_1 = 0.2,$$
$$k_2 = 0.205$$
$$k_3 = 0.20525$$
$$k_4 = 0.21053.$$

Hence

$$y(0.1) = 2 + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) = 2.2052.$$

Proceeding similarly, we obtain $y(0.2) = 2.4214$.

Example 8.9 Given $dy/dx = 1 + y^2$, where $y = 0$ when $x = 0$, find $y(0.2)$, $y(0.4)$ and $y(0.6)$.

We take $h = 0.2$. With $x_0 = y_0 = 0$, we obtain from (8.21a) and (8.21b),

$$k_1 = 0.2,$$
$$k_2 = 0.2\,(1.01) = 0.202,$$
$$k_3 = 0.2\,(1 + 0.010201) = 0.20204,$$
$$k_4 = 0.2\,(1 + 0.040820) = 0.20816,$$

and

$$y(0.2) = 0 + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) = 0.2027,$$

which is correct to four decimal places.

To compute $y(0.4)$, we take $x_0 = 0.2$, $y_0 = 0.2027$ and $h = 0.2$. With these values, Eqs. (8.21a) and (8.21b) give

$$k_1 = 0.2\,[1 + (0.2027)^2] = 0.2082,$$

$$k_2 = 0.2\,[1 + (0.3068)^2] = 0.2188,$$

$$k_3 = 0.2\,[1 + (0.3121)^2] = 0.2195,$$

$$k_4 = 0.2\,[1 + (0.4222)^2] = 0.2356,$$

and

$$y(0.4) = 0.2027 + 0.2201 = 0.4228,$$

correct to four decimal places.

Finally, taking $x_0 = 0.4$, $y_0 = 0.4228$ and $h = 0.2$, and proceeding as above, we obtain $y(0.6) = 0.6841$.

Example 8.10 We consider the initial value problem $y' = 3x + y/2$ with the condition $y(0) = 1$.

The following table gives the values of $y(0.2)$ by different methods, the exact value being 1.16722193. It is seen that the *fourth-order* Runge–Kutta method gives the accurate value for $h = 0.05$.

Method	*h*	*Computed value*
Euler	0.2	1.100 000 00
	0.1	1.132 500 00
	0.05	1.149 567 58
Modified Euler	0.2	1.100 000 00
	0.1	1.150 000 00
	0.05	1.162 862 42
Fourth-order Runge–Kutta	0.2	1.167 220 83
	0.1	1.167 221 86
	0.05	1.167 221 93

8.6 PREDICTOR–CORRECTOR METHODS

In the methods described so far, to solve a differential equation over a single interval, say from $x = x_n$ to $x = x_{n+1}$, we required information only at the beginning of the interval, i.e. at $x = x_n$. *Predictor–corrector* methods are the ones which require function values at x_n, x_{n-1}, x_{n-2}, … for the computation of the function value at x_{n+1} A *predictor* formula is used to predict the value of y at x_{n+1} and then a *corrector* formula is used to improve the value of y_{n+1}.

In Section 8.6.1 we derive Predictor–corrector formulae which use backward differences and in Section 8.6.2 we describe Milne's method which uses forward differences.

Hence, Eq. (8.41) gives

$$4m_0 + 8m_1 - 24(s_1 - s_0) = 3 + \frac{1}{2}\left(\frac{3}{2} + \frac{1}{2}s_1\right)$$

and

$$4m_1 + 8m_2 - 24(s_2 - s_1) = 3 + \frac{1}{2}\left(3 + \frac{1}{2}s_2\right)$$

Since $m_0 = 1/2,\ m_1 = 3/2 + s_1/2$ and $m_2 = 3 + s_2/2,$ the above equations give on simplification

$$s_1 = 1.691358 \quad \text{and} \quad s_2 = 3.430879.$$

The errors in these solutions are given by 0.000972 and 0.002497, respectively. It can be shown that, under certain conditions, the spline method gives $O(h^4)$ convergence and compares well with the multi-step Milne's method. For details, the reader is referred to Patricio [1978].

8.8 SIMULTANEOUS AND HIGHER-ORDER EQUATIONS

We consider the two equations

$$\frac{dx}{dt} = f(t, x, y) \quad \text{and} \quad \frac{dy}{dt} = \phi(t, x, y) \tag{8.42}$$

with the initial conditions $x = x_0$ and $y = y_0$, when $t = t_0$. Assuming that $\Delta t = h,$ $\Delta x = k,$ and $\Delta y = l,$ the fourth-order Runge–Kutta method gives

$$\left.\begin{aligned}
k_1 &= hf(t_0, x_0, y_0);\\
l_1 &= h\phi(t_0, x_0, y_0);\\
k_2 &= hf\left(t_0 + \frac{1}{2}h, x_0 + \frac{1}{2}k_1, y_0 + \frac{1}{2}l_1\right);\\
l_2 &= h\phi\left(t_0 + \frac{1}{2}h, x_0 + \frac{1}{2}k_1, y_0 + \frac{1}{2}l_1\right);\\
k_3 &= hf\left(t_0 + \frac{1}{2}h, x_0 + \frac{1}{2}k_2, y_0 + \frac{1}{2}l_2\right);\\
l_3 &= h\phi\left(t_0 + \frac{1}{2}h, x_0 + \frac{1}{2}k_2, y_0 + \frac{1}{2}l_2\right);\\
k_4 &= hf(t_0 + h, x_0 + k_3, y_0 + l_3);\\
l_4 &= h\phi(t_0 + h, x_0 + k_3, y_0 + l_3);\\
x_1 &= x_0 + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)\\
y_1 &= y_0 + \frac{1}{6}(l_1 + 2l_2 + 2l_3 + l_4).
\end{aligned}\right\} \tag{8.43}$$

In a similar manner, one can extend the Taylor series method or Picard's method to the system (8.42). The extension of the Runge–Kutta method to a system of n equations is quite straightforward.

We now consider the second-order differential equation

$$y'' = F(x, y, y') \tag{8.44a}$$

with the initial conditions

$$y(x_0) = y_0 \quad \text{and} \quad y'(x_0) = y_0'. \tag{8.45a}$$

By setting $z = y'$, the problem given in Eqs. (8.44a) and (8.45a) can be reduced to the problem of solving the system

$$y' = z \quad \text{and} \quad z' = F(x, y, z) \tag{8.44b}$$

with the conditions

$$y(x_0) = y_0 \quad \text{and} \quad z(x_0) = y_0' \tag{8.45b}$$

which can be solved by the method described above. Similarly, any higher-order differential equation, in which we can solve for the highest derivative, can be reduced to a system of first-order differential equations.

8.9 SOME GENERAL REMARKS

In the preceding sections, we have given a brief discussion of some well-known methods for the numerical solution of an ordinary differential equation satisfying certain given initial conditions. If the solution is required over a wider range, it is important to get the starting values as accurately as possible by one of the methods described.

It is outside the scope of this book to present a comprehensive review of the different methods described in this text for the numerical solution of differential equations, but the following points are relevant to the methods discussed.

The Taylor's series method suffers from the serious disadvantage that all the higher derivatives of $f(x, y)$ [see Eqs. (8.1)] must exist and that h should be small such that successive terms of the series diminish quite rapidly. Likewise, in the modified Euler method, the value of h should be so small that one or two applications of the iteration formula (8.14) will give the final result for that value of h. The Picard method has probably little practical value because of the difficulty in performing the successive integrations.

Although laborious, the Runge–Kutta method is the most widely used one since it gives reliable starting values and is particularly suitable when the computation of higher derivatives is complicated. When the starting values have been found, the computations for the rest of the interval can be continued by means of the predictor–corrector methods.

Chapter 5
Ordinary Differential Equations: Initial Value Problems

5.1 Introduction

In this chapter we will introduce common numeric methods designed to solve *initial value problems*. Within our discussion of the KEPLER problem in the previous chapter we introduced four concepts, namely the implicit EULER method, the explicit EULER method, the implicit midpoint rule, and we mentioned the symplectic EULER method. In this chapter we plan to put these methods into a more general context and to discuss more advanced techniques.

Let us define the problem: We consider initial value problems of the form

$$\begin{cases} \dot{y}(t) = f(y,t), \\ y(0) = y_0, \end{cases} \tag{5.1}$$

where $y(t) \equiv y$ is an n-dimensional vector and y_0 is referred to as the *initial value* of y. Let us make some remarks about the form of Eq. (5.1).

(i) We note that by posing Eq. (5.1), we assume that the differential equation is *explicit* in $\dot{y}$; i.e. initial value problems of the form

$$\begin{cases} G(\dot{y}) = f(y,t), \\ y(0) = y_0, \end{cases} \tag{5.2}$$

are only considered if $G(\dot{y})$ is analytically invertible. For instance, we will not deal with differential equations of the form

$$\dot{y} + \log(\dot{y}) = 1. \tag{5.3}$$

(ii) We note that Eq. (5.1) is a *first order* differential equation in y. However, this is in fact not a restriction since we can transform every explicit differential equation of order n into a coupled set of explicit first order differential equations. Let us demonstrate this. We regard an explicit differential equation of the form

B. A. Stickler and E. Schachinger, *Basic Concepts in Computational Physics*,
DOI: 10.1007/978-3-319-02435-6_5, © Springer International Publishing Switzerland 2014

$$y^{(n)} = f(t; y, \dot{y}, \ddot{y}, \ldots, y^{(n-1)}), \tag{5.4}$$

where we defined $y^{(k)} \equiv \frac{\mathrm{d}^k}{\mathrm{d}t^k} y$. This equation is equivalent to the set

$$\begin{aligned} \dot{y}_1 &= y_2, \\ \dot{y}_2 &= y_3, \\ &\vdots \quad \vdots \\ \dot{y}_{n-1} &= y_n, \\ \dot{y}_n &= f(t, y_1, y_2, \ldots, y_n), \end{aligned} \tag{5.5}$$

which can be written as Eq. (5.1). Hence, we can attenuate the criterion discussed in point (i), i.e. that the differential equation has to be explicit in $\dot{y}$, to the criterion that the differential equation of order n has to be explicit in the n-th derivative of y, namely $y^{(n)}$.

There is another point required to be discussed before moving on. The numerical treatment of initial value problems is of inestimable value in physics because many differential equations, which appear unspectacular at first glance, cannot be solved analytically. For instance, consider a first order differential equation:

$$\dot{y} = t^2 + y^2. \tag{5.6}$$

Although this equation appears to be simple, one has to rely on numerical methods in order to obtain a solution. However, Eq. (5.6) is not *well posed* since the solution is ambiguous as long as no initial values are given. A numerical solution is only possible if the problem is completely defined. In many cases, one uses numerical methods although the problem is solvable with the help of analytic methods, simply because the solution would be too complicated. A numerical approach might be justified, however, one should always remember that [1], quote:

"Numerical methods are no excuse for poor analysis."

This chapter is augmented by a chapter on the double pendulum, which will serve as a demonstration of the applicability of RUNGE-KUTTA methods and by a chapter on molecular dynamics which will demonstrate the applicability of the leap-frog algorithm.

5.2 Simple Integrators

We start by reintroducing the methods already discussed in the previous chapter. Again, we discretize the time coordinate t via the relation $t_n = t_0 + n\Delta t$ and define $f_n \equiv f(t_n)$ accordingly. In the following we will refrain from noting the initial condition explicitly for a more compact notation. We investigate Eq. (5.1) at some particular time t_n:

$$\dot{y}_n = f(y_n, t_n). \tag{5.7}$$

Integrating both sides of (5.7) over the interval $[t_n, t_{n+1}]$ gives

$$y_{n+1} = y_n + \int_{t_n}^{t_{n+1}} \mathrm{d}t' f[y(t'), t']. \tag{5.8}$$

Note that Eq. (5.8) is exact and it will be our starting point in the discussion of several paths to a numeric solution of initial value problems. These solutions will be based on an approximation of the integral on the right hand side of Eq. (5.8) with the help of the methods already discussed in Chap. 3.

In the following we list four of the best known simple integration methods for initial value problems:

(1)

Applying the forward rectangular rule (3.9) to Eq. (5.8) yields

$$y_{n+1} = y_n + f(y_n, t_n)\Delta t + \mathcal{O}(\Delta t^2), \tag{5.9}$$

which is the explicit EULER method we encountered already in Sect. 4.3. This method is also referred to as the *forward* EULER *method* . In accordance to the forward rectangular rule, the leading term of the error of this method is proportional to Δt^2 as was pointed out in Sect. 3.2.

(2)

We use the backward rectangular rule (3.10) in Eq. (5.8) and obtain

$$y_{n+1} = y_n + f(y_{n+1}, t_{n+1})\Delta t + \mathcal{O}(\Delta t^2), \tag{5.10}$$

which is the implicit EULER method, also referred to as *backward* EULER *method*. As already highlighted in Sect. 4.3, it may be necessary to solve Eq. (5.10) numerically for y_{n+1} (Some notes on the numeric solution of non-linear equations can be found in Appendix B).

(3)

The central rectangular rule (3.13) approximates Eq. (5.8) by

$$y_{n+1} = y_n + f(y_{n+\frac{1}{2}}, t_{n+\frac{1}{2}})\Delta t + \mathcal{O}(\Delta t^3), \tag{5.11}$$

and we rewrite this equation in the form:

$$y_{n+1} = y_{n-1} + 2f(y_n, t_n)\Delta t + \mathcal{O}(\Delta t^3). \tag{5.12}$$

This method is sometimes referred to as the *leap-frog* routine or STÖRMER-VERLET method. We will come back to this point in Chap. 7. Note that the approximation

$$y_{n+\frac{1}{2}} \approx \frac{y_n + y_{n+1}}{2}, \tag{5.13}$$

gives the implicit midpoint rule as it was introduced in Sect. 4.3.

(4)

Employing the trapezoidal rule (3.15) in an approximation to Eq. (5.8) yields

$$y_{n+1} = y_n + \frac{\Delta t}{2}\left[f(y_n, t_n) + f(y_{n+1}, t_{n+1})\right] + \mathcal{O}(\Delta t^3). \tag{5.14}$$

This is an implicit method which has to be solved for y_{n+1}. It is generally known as the CRANK-NICOLSON method or simply as *trapezoidal method*.

Methods (1), (2), and (4) are also known as one-step methods, since only function values at times t_n and t_{n+1} are used to propagate in time. In contrast, the leap-frog method is already a *multi-step* method since three different times appear in the expression. Basically, there are three different strategies to improve these rather simple methods:

- TAYLOR series methods: Use more terms in the TAYLOR expansion of y_{n+1}.
- Linear Multi-Step methods: Use data from previous time steps y_k, $k < n$ in order to cancel terms in the truncation error.
- RUNGE-KUTTA method: Use intermediate points within one time step.

We will briefly discuss the first two alternatives and then turn our attention to the RUNGE- KUTTA methods in the next section.

Taylor Series Methods

From Chap. 2 we are already familiar with the TAYLOR expansion (2.7) of the function y_{n+1} around the point y_n,

$$y_{n+1} = y_n + \Delta t\dot{y}_n + \frac{\Delta t^2}{2}\ddot{y}_n + \mathcal{O}(\Delta t^3). \tag{5.15}$$

We insert Eq. (5.7) into Eq. (5.15) and obtain

$$y_{n+1} = y_n + \Delta t f(y_n, t_n) + \frac{\Delta t^2}{2}\ddot{y}_n + \mathcal{O}(\Delta t^3). \tag{5.16}$$

So far nothing has been gained since the truncation error is still proportional to Δt^2. However, calculating $\ddot{y}_n$ with the help of Eq. (5.7) gives

$$\ddot{y}_n = \frac{\mathrm{d}}{\mathrm{d}t} f(y_n, t_n) = \dot{f}(y_n, t_n) + f'(y_n, t_n)\dot{y}_n = \dot{f}(y_n, t_n) + f'(y_n, t_n) f(y_n, t_n), \tag{5.17}$$

and this results together with Eq. (5.16) in:

$$y_{n+1} = y_n + \Delta t f(y_n, t_n) + \frac{\Delta t^2}{2}\left[\dot{f}(y_n, t_n) + f'(y_n, t_n) f(y_n, t_n)\right] + \mathcal{O}(\Delta t^3). \tag{5.18}$$

This manipulation reduced the local truncation error to orders of Δt^3. The derivatives of $f(y_n, t_n)$, $f'(y_n, t_n)$ and $\dot{f}(y_n, t_n)$ can be approximated with the help of the methods discussed in Chap. 2, if an analytic differentiation is not feasible.

The above procedure can be repeated up to arbitrary order in the TAYLOR expansion (5.15).

Linear Multi-Step Methods

A k-th order linear multi-step method is defined by the approximation

$$y_{n+1} = \sum_{j=0}^{k} a_j y_{n-j} + \Delta t \sum_{j=0}^{k+1} b_i f(y_{n+1-j}, t_{n+1-j}), \tag{5.19}$$

of Eq. (5.8). The coefficients a_j and b_j have to be determined in such a way that the truncation error is reduced. Two of the best known techniques are the so called second order ADAMS–BASHFORD methods

$$y_{n+1} = y_n + \frac{\Delta t}{2}\left[3f(y_n, t_n) - f(y_{n-1}, t_{n-1})\right] \tag{5.20}$$

and the second order rule (*backward differentiation formula*)

$$y_{n+1} = \frac{1}{3}\left[4y_n - y_{n-1} + \frac{\Delta t}{2} f(y_{n+1}, t_{n+1})\right]. \tag{5.21}$$

We note in passing that the backward differentiation formula of arbitrary order can easily be obtained with the help of the operator technique introduced in Sect. 2.4, Eq. (2.30). One simply replaces the first derivative on the left hand side by the function $f(y_n, t_n)$ according to Eq. (5.7) and calculates the backward difference series on the right hand side to arbitrary order.

In many cases, multi-step methods are based on the interpolation of previously computed values y_k by LAGRANGE polynomials. This interpolation is then inserted into Eq. (5.8) and integrated. However, a detailed discussion of such procedures is beyond the scope of this book. The interested reader is referred to Refs. [2, 3].

Nevertheless, let us make one last point. We note that Eq. (5.19) is explicit for $b_0 = 0$ and implicit for $b_0 \neq 0$. In many numerical realizations one combines implicit and explicit multi-step methods in such a way that the explicit result (solve Eq. (5.19) with $b_0 = 0$) is used as a guess to solve the implicit equation (solve Eq. (5.19) with $b_0 \neq 0$). Hence, the explicit method *predicts* the value y_{n+1} and the implicit method *corrects* it. Such methods yield very good results and are commonly referred to as *predictor–corrector* methods [4].

5.3 RUNGE-KUTTA Methods

In contrast to linear multi-step methods, the idea in RUNGE-KUTTA methods is to improve the accuracy by calculating intermediate grid-points within the interval $[t_n, t_{n+1}]$. We note that the approximation (5.11) resulting from the central rectangular rule is already such a method since the function value $y_{n+\frac{1}{2}}$ at the grid-point $t_{n+\frac{1}{2}} = t_n + \frac{\Delta t}{2}$ is taken into account. We investigate this in more detail and rewrite Eq. (5.11):

$$y_{n+1} = y_n + f(y_{n+\frac{1}{2}}, t_{n+\frac{1}{2}})\Delta t + \mathcal{O}(\Delta t^3). \tag{5.22}$$

We now have to find appropriate approximations to $y_{n+\frac{1}{2}}$ which will increase the accuracy of Eq. (5.11). Our first choice is to replace $y_{n+\frac{1}{2}}$ with the help of the explicit EULER method, Eq. (5.9),

$$y_{n+\frac{1}{2}} = y_n + \frac{\Delta t}{2}\dot{y}_n = y_n + \frac{\Delta t}{2} f(y_n, t_n), \tag{5.23}$$

which, inserted into Eq. (5.22) yields

$$y_{n+1} = y_n + f\left[y_n + \frac{\Delta t}{2} f(y_n, t_n), t_n + \frac{\Delta t}{2}\right]\Delta t + \mathcal{O}(\Delta t^2). \tag{5.24}$$

We note that Eq. (5.24) is referred to as the *explicit midpoint rule*. In analogy we could have approximated $y_{n+\frac{1}{2}}$ with the help of the implicit EULER method (5.10) which yields

$$y_{n+1} = y_n + f\left[y_n + \frac{\Delta t}{2} f(y_{n+1}, t_{n+1}), t_n + \frac{\Delta t}{2}\right]\Delta t + \mathcal{O}(\Delta t^2). \tag{5.25}$$

This equation is referred to as the *implicit midpoint rule*. Let us explain how we obtain an estimate for the error in Eqs. (5.24) and (5.25). In case of Eq. (5.24) we investigate the term

$$y_{n+1} - y_n - f\left[y_n + \frac{\Delta t}{2} f(y_n, t_n), t_n + \frac{\Delta t}{2}\right] \Delta t.$$

The TAYLOR expansion of y_{n+1} and $f(\cdot)$ around the point $\Delta t = 0$ yields

$$\Delta t \left[\dot{y}_n - f(y_n, t_n)\right] + \frac{\Delta t^2}{2} \left[\ddot{y} - \dot{f}(y_n, t_n) - f'(y_n, t_n)\dot{y}_n\right] + \cdots . \tag{5.26}$$

We observe that the first term cancels because of Eq. (5.7). Consequently, the error is of order Δt^2. A similar argument holds for Eq. (5.25).

Let us introduce a more convenient notation for the above examples before we concentrate on a more general topic. It is presented in algorithmic form, i.e. it defines the sequence in which one should calculate the various terms. This is convenient for two reasons, first of all it increases the readability of complex methods such as Eq. (5.25) and, secondly, it can be easily identified which part of the method involves an implicit step which has to be solved separately for the corresponding variable. For this purpose let us introduce variables Y_i of some index $i \geq 1$ and we use a simple example to illustrate this notation. Consider the explicit EULER method (5.9). It can be written as

$$\begin{aligned} Y_1 &= y_n, \\ y_{n+1} &= y_n + f(Y_1, t_n)\Delta t. \end{aligned} \tag{5.27}$$

In a similar fashion we write the implicit EULER method (5.10) as

$$\begin{aligned} Y_1 &= y_n + f(Y_1, t_{n+1})\Delta t, \\ y_{n+1} &= y_n + f(Y_1, t_{n+1})\Delta t. \end{aligned} \tag{5.28}$$

It is understood that the first equation of (5.28) has to be solved for Y_1 first and this result is then plugged into the second equation in order to obtain y_{n+1}. One further example: the CRANK–NICOLSON (5.14) method can be rewritten as

$$\begin{aligned} Y_1 &= y_n, \\ Y_2 &= y_n + \frac{\Delta t}{2}\left[f(Y_1, t_n) + f(Y_2, t_{n+1})\right], \\ y_{n+1} &= y_n + \frac{\Delta t}{2}\left[f(Y_1, t_n) + f(Y_2, t_{n+1})\right], \end{aligned} \tag{5.29}$$

where the second equation is to be solved for Y_2 in the second step.

In analogy, the algorithmic form of the explicit midpoint rule (5.24) is defined as

$$
\begin{aligned}
Y_1 &= y_n, \\
Y_2 &= y_n + \frac{\Delta t}{2} f\left(Y_1, t_n + \frac{\Delta t}{2}\right), \\
y_{n+1} &= y_n + \frac{\Delta t}{2} f\left(Y_2, t_n + \frac{\Delta t}{2}\right),
\end{aligned} \tag{5.30}
$$

and we find for the implicit midpoint rule (5.25):

$$
\begin{aligned}
Y_1 &= y_n + \frac{\Delta t}{2} f\left(Y_1, t_n + \frac{\Delta t}{2}\right), \\
y_{n+1} &= y_n + \Delta t f\left(Y_1, t_n + \frac{\Delta t}{2}\right).
\end{aligned} \tag{5.31}
$$

The above algorithms are all examples of the so called RUNGE-KUTTA methods. We introduce the general representation of a d-stage RUNGE-KUTTA method:

$$
\begin{aligned}
Y_i &= y_n + \Delta t \sum_{j=1}^{d} a_{ij} f\left(Y_j, t_n + c_j \Delta t\right), \qquad i = 1, \ldots, d, \\
y_{n+1} &= y_n + \Delta t \sum_{j=1}^{d} b_i f\left(Y_j, t_n + c_j \Delta t\right).
\end{aligned} \tag{5.32}
$$

We note that Eq. (5.32) it is completely determined by the coefficients a_{ij}, b_j and c_j. In particular $a = \{a_{ij}\}$ is a $d \times d$ matrix, while $b = \{b_j\}$ and $c = \{c_j\}$ are d dimensional vectors.

BUTCHER tableaus are a very useful tool to characterize such methods. They provide a structured representation of the coefficient matrix a and the coefficient vectors b and c:

$$
\begin{array}{c|cccc}
c_1 & a_{11} & a_{12} & \ldots & a_{1d} \\
c_2 & a_{21} & a_{22} & \ldots & a_{2d} \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
c_d & a_{d1} & a_{d2} & \ldots & a_{dd} \\
\hline
 & b_1 & b_2 & \ldots & b_d
\end{array} \tag{5.33}
$$

We note that the RUNGE-KUTTA method (5.32) or (5.33) is explicit if the matrix a is zero on and above the diagonal, i.e. $a_{ij} = 0$ for $j \geq i$. Let us rewrite all the methods described here in the form of BUTCHER tableaus:

Explicit EULER:

$$\begin{array}{c|c} 0 & 0 \\ \hline & 1 \end{array} \tag{5.34}$$

Implicit EULER:

$$\begin{array}{c|c} 1 & 1 \\ \hline & 1 \end{array} \tag{5.35}$$

CRANK- NICOLSON:

$$\begin{array}{c|cc} 0 & 0 & 0 \\ 1 & \frac{1}{2} & \frac{1}{2} \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array} \tag{5.36}$$

Explicit Midpoint:

$$\begin{array}{c|cc} 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array} \tag{5.37}$$

Implicit Midpoint:

$$\begin{array}{c|c} \frac{1}{2} & \frac{1}{2} \\ \hline & 1 \end{array} \tag{5.38}$$

With the help of RUNGE–KUTTA methods of the general form (5.32) one can develop methods of arbitrary accuracy. One of the most popular methods is the explicit four stage method (we will call it *e-RK-4*) which is defined by the algorithm:

$$\begin{aligned} Y_1 &= y_n, \\ Y_2 &= y_n + \frac{\Delta t}{2} f(Y_1, t_n), \\ Y_3 &= y_n + \frac{\Delta t}{2} f\left(Y_2, t_n + \frac{\Delta t}{2}\right), \\ Y_4 &= y_n + \Delta t f\left(Y_3, t_n + \frac{\Delta t}{2}\right), \\ y_{n+1} &= y_n + \frac{\Delta t}{6}\left[f(Y_1, t_n) + 2f\left(Y_2, t_n + \frac{\Delta t}{2}\right)\right. \end{aligned}$$

$$+2f\left(Y_3, t_n + \frac{\Delta t}{2}\right) + f(Y_4, t_n)\Big]. \tag{5.39}$$

This method is an analogue to the SIMPSON rule of numerical integration as discussed in Sect. 3.4. However, a detailed compilation of the coefficient array a and coefficient vectors b, and c is quite complicated. A closer inspection reveals that the methodological error of this method behaves as Δt^5. The algorithm *e-RK-4*, Eq. (5.39), is represented by a BUTCHER tableau of the form

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array} \tag{5.40}$$

Another quite popular method is given by the Butcher tableau

$$\begin{array}{c|cc} \frac{1}{2} - \frac{\sqrt{3}}{6} & \frac{1}{4} & \frac{1}{4} - \frac{\sqrt{3}}{6} \\ \frac{1}{2} + \frac{\sqrt{3}}{6} & \frac{1}{4} + \frac{\sqrt{3}}{6} & \frac{1}{4} \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array} \tag{5.41}$$

We note that this method is implicit and mention that it corresponds to the two point GAUSS-LEGENDRE quadrature of Sect. 3.6.

A further improvement of implicit RUNGE-KUTTA methods can be achieved by *choosing* the Y_i in such a way that they correspond to solutions of the differential equation (5.7) at intermediate time steps. The intermediate time steps at which one wants to reproduce the function are referred to as *collocation points*. At these points the functions are approximated by interpolation on the basis of LAGRANGE polynomials, which can easily be integrated analytically. However, the discussion of such collocation methods [4] is far beyond the scope of this book.

In general RUNGE-KUTTA methods are very useful. However one always has to keep in mind that there could be better methods for the problem at hand. Let us close this section with a quote from the book by PRESS et al. [5]:

"*For many scientific users, fourth-order Runge-Kutta is not just the first word on ODE integrators, but the last word as well. In fact, you can get pretty far on this old workhorse, especially if you combine it with an adaptive step-size algorithm. Keep in mind, however, that the old workhorse's last trip may well take you to the poorhouse: Bulirsch-Stoer or predictor-corrector methods can be very much more efficient for problems where high accuracy is a requirement. Those methods are the high-strung racehorses. Runge-Kutta is for ploughing the fields.*"

5.4 Hamiltonian Systems: Symplectic Integrators

Let us define a symplectic integrator as a numerical integration in which the mapping

$$\Phi_{\Delta t} : y_n \mapsto y_{n+1}, \tag{5.42}$$

is symplectic. Here $\Phi_{\Delta t}$ is referred to as the *numerical flow* of the method. If we regard the initial value problem (5.1) we can define in an analogous way the *flow of the system* φ_t as

$$\varphi_t(y_0) = y(t). \tag{5.43}$$

For instance, if we consider the initial value problem

$$\begin{cases} \dot{y} = Ay, \\ y(0) = y_0, \end{cases} \tag{5.44}$$

where $y \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n\times n}$, then the flow of the system φ_t is given by:

$$\varphi_t(y_0) = \exp(At)y_0. \tag{5.45}$$

On the other hand, if we regard two vectors $v, w \in \mathbb{R}^2$, we can express the area ω of the parallelogram spanned by these vectors as

$$\omega(v, w) = \det(vw) = v \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} w = ad - bc, \tag{5.46}$$

where we put $v = (a, b)^T$ and $w = (c, d)^T$. More generally, if $v, w \in \mathbb{R}^{2d}$, we have

$$\omega(v, w) = v \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix} w \equiv vJw, \tag{5.47}$$

where I is the $d \times d$ dimensional unity matrix. Hence (5.47) represents the sum of the projected areas of the form

$$\det \begin{pmatrix} v_i & w_i \\ v_{i+d} & w_{i+d} \end{pmatrix}. \tag{5.48}$$

If we regard a mapping $M : \mathbb{R}^{2d} \mapsto \mathbb{R}^{2d}$ and require that

$$\omega(Mv, Mw) = \omega(v, w), \tag{5.49}$$

i.e. the area is preserved, we obtain the condition that

$$M^T JM = J, \tag{5.50}$$

which is equivalent to $\det(M) = 1$. Finally, a differentiable mapping $f : \mathbb{R}^{2d} \mapsto \mathbb{R}^{2d}$ is referred to as *symplectic* if the linear mapping $f'(x)$ (JACOBI matrix) conserves ω for all $x \in \mathbb{R}^{2d}$. One can easily prove that the flow of Hamiltonian systems (energy conserving) is symplectic, i.e. area preserving in phase space. Every Hamiltonian system is characterized by its HAMILTON function $H(p, q)$ and the corresponding HAMILTON equations of motion:

$$\dot{p} = -\nabla_q H(p, q) \qquad \text{and } \dot{q} = \nabla_p H(p, q). \tag{5.51}$$

We define the flow of the system via

$$\varphi_t(x_0) = x(t), \tag{5.52}$$

where

$$x_0 = \begin{pmatrix} p_0 \\ q_0 \end{pmatrix} \quad \text{and} \quad x(t) = \begin{pmatrix} p(t) \\ q(t) \end{pmatrix}. \tag{5.53}$$

Hence we rewrite (5.51) as

$$\dot{x} = J^{-1} \nabla_x H(x), \tag{5.54}$$

and note that $x \equiv x(t, x_0)$ is a function of time and initial conditions. In a next step we define the Jacobian of the flow via

$$P_t(x_0) = \nabla_{x_0} \phi_t(x_0), \tag{5.55}$$

and calculate

$$\begin{aligned} \dot{P}_t(x_0) &= \nabla_{x_0} \dot{x} \\ &= J^{-1} \nabla_{x_0} \nabla_x H(x) \\ &= J^{-1} \Delta_x H(x) \nabla_{x_0} x \\ &= J^{-1} \Delta_x H(x) P_t(x_0) \\ &= \begin{pmatrix} -\nabla_{qp} H(p, q) & -\nabla_{qq} H(p, q) \\ \nabla_{pp} H(p, q) & \nabla_{pq} H(p, q) \end{pmatrix} P_t(x_0). \end{aligned} \tag{5.56}$$

Hence, P_t is given by the solution of the equation

$$\dot{P}_t = J^{-1} \Delta_x H(x) P_t. \tag{5.57}$$

Symplecticity ensures that the area

$$P_t^T J P_t = \text{const}, \tag{5.58}$$

which can be verified by calculating $\frac{\mathrm{d}}{\mathrm{d}t} \left(P_t^T J P_t \right)$ where we keep in mind that $J^T = -J$. Hence,

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}t}P_t^T J P_t &= \dot{P}_t^T J P_t + P_t^T J \dot{P}_t \\ &= P_t^T \Delta_x H(x)(J^{-1})^T J P_t + P_t^T J J^{-1} \Delta_x H(x) P_t \\ &= 0, \end{aligned} \tag{5.59}$$

if the HAMILTON function is conserved, i.e.

$$\frac{\partial}{\partial t} H(p, q) \stackrel{!}{=} 0. \tag{5.60}$$

This means that the flow of a Hamiltonian system is symplectic, i.e. area preserving in phase space.

Since this conservation law is violated by methods like *e-RK-4* or explicit EULER, one introduces so called *symplectic integrators*, which have been particularly designed as a remedy to this shortcoming. A detailed investigation of these techniques is far too engaged for this book. The interested reader is referred to Refs. [6–9].

However, we provide a list of the most important integrators.

Symplectic EULER

$$q_{n+1} = q_n + a(q_n, p_{n+1})\Delta t, \tag{5.61a}$$

$$p_{n+1} = p_n + b(q_n, p_{n+1})\Delta t. \tag{5.61b}$$

Here $a(p, q) = \nabla_p H(p, q)$ and $b(p, q) = -\nabla_q H(p, q)$ have already been defined in Sect. 4.3.

Symplectic RUNGE–KUTTA

It can be demonstrated that a RUNGE-KUTTA method is symplectic if the coefficients fulfill

$$b_i a_{ij} + b_j a_{ji} = b_i b_j, \tag{5.62}$$

for all i, j [7]. This is a property of the collocation methods based on GAUSS points c_i.

5.5 An Example: The KEPLER Problem, Revisited

It has already been discussed in Sect. 4.3 that the HAMILTON function of this system takes on the form

$$H(p, q) = \frac{1}{2}\left(p_1^2 + p_2^2\right) - \frac{1}{\sqrt{q_1^2 + q_2^2}}, \tag{5.63}$$

and HAMILTON's equations of motion read

$$\dot{p}_1 = -\nabla_{q_1} H(p,q) = -\frac{q_1}{(q_1^2+q_2^2)^{\frac{3}{2}}}, \tag{5.64a}$$

$$\dot{p}_2 = -\nabla_{q_2} H(p,q) = -\frac{q_2}{(q_1^2+q_2^2)^{\frac{3}{2}}}, \tag{5.64b}$$

$$\dot{q}_1 = \nabla_{p_1} H(p,q) = p_1, \tag{5.64c}$$

$$\dot{q}_2 = \nabla_{p_2} H(p,q) = p_2. \tag{5.64d}$$

We now introduce the time instances $t_n = t_0 + n\Delta t$ and define $q_i^n \equiv q_i(t_n)$ and $p_i^n \equiv p_i(t_n)$ for $i = 1, 2$. In the following we give the discretized recursion relation for three different methods, namely explicit EULER, implicit EULER, and symplectic EULER.

Explicit EULER

In case of the explicit EULER method we have simple recursion relations

$$p_1^{n+1} = p_1^n - \frac{q_1^n \Delta t}{[(q_1^n)^2 + (q_2^n)^2]^{\frac{3}{2}}}, \tag{5.65a}$$

$$p_2^{n+1} = p_2^n - \frac{q_2^n \Delta t}{[(q_1^n)^2 + (q_2^n)^2]^{\frac{3}{2}}}, \tag{5.65b}$$

$$q_1^{n+1} = q_1^n + p_1^n \Delta t, \tag{5.65c}$$

$$q_2^{n+1} = q_2^n + p_2^n \Delta t. \tag{5.65d}$$

Implicit EULER

We obtain the implicit equations

$$p_1^{n+1} = p_1^n - \frac{q_1^{n+1} \Delta t}{[(q_1^{n+1})^2 + (q_2^{n+1})^2]^{\frac{3}{2}}}, \tag{5.66a}$$

$$p_2^{n+1} = p_2^n - \frac{q_2^{n+1} \Delta t}{[(q_1^{n+1})^2 + (q_2^{n+1})^2]^{\frac{3}{2}}}, \tag{5.66b}$$

$$q_1^{n+1} = q_1^n + p_1^{n+1} \Delta t, \tag{5.66c}$$

$$q_2^{n+1} = q_2^n + p_2^{n+1} \Delta t. \tag{5.66d}$$

These implicit equations can be solved, for instance, by the use of the NEWTON method discussed in Appendix B.

Symplectic EULER

Employing Eqs. (5.61) gives

$$p_1^{n+1} = p_1^n - \frac{q_1^n \Delta t}{[(q_1^n)^2 + (q_2^n)^2]^{\frac{3}{2}}}, \tag{5.67a}$$

$$p_2^{n+1} = p_2^n - \frac{q_2^n \Delta t}{[(q_1^n)^2 + (q_2^n)^2]^{\frac{3}{2}}}, \tag{5.67b}$$

$$q_1^{n+1} = q_1^n + p_1^{n+1} \Delta t, \tag{5.67c}$$

$$q_2^{n+1} = q_2^n + p_2^{n+1} \Delta t. \tag{5.67d}$$

These implicit equations can be solved analytically and we obtain

$$p_1^{n+1} = p_1^n - \frac{q_1^n \Delta t}{[(q_1^n)^2 + (q_2^n)^2]^{\frac{3}{2}}}, \tag{5.68a}$$

$$p_2^{n+1} = p_2^n - \frac{q_2^n \Delta t}{[(q_1^n)^2 + (q_2^n)^2]^{\frac{3}{2}}}, \tag{5.68b}$$

$$q_1^{n+1} = q_1^n + p_1^n \Delta t - \frac{q_1^n \Delta t^2}{[(q_1^n)^2 + (q_2^n)^2]^{\frac{3}{2}}}, \tag{5.68c}$$

$$q_2^{n+1} = q_2^n + p_2^n \Delta t - \frac{q_2^n \Delta t^2}{[(q_1^n)^2 + (q_2^n)^2]^{\frac{3}{2}}}. \tag{5.68d}$$

A second possibility of the symplectic EULER is given by Eq. (4.41). It reads

$$p_1^{n+1} = p_1^n - \frac{q_1^{n+1} \Delta t}{[(q_1^{n+1})^2 + (q_2^{n+1})^2]^{\frac{3}{2}}}, \tag{5.69a}$$

$$p_2^{n+1} = p_2^n - \frac{q_2^{n+1} \Delta t}{[(q_1^{n+1})^2 + (q_2^{n+1})^2]^{\frac{3}{2}}}, \tag{5.69b}$$

$$q_1^{n+1} = q_1^n + p_1^n \Delta t, \tag{5.69c}$$

$$q_2^{n+1} = q_2^n + p_2^n \Delta t. \tag{5.69d}$$

The trajectories calculated using these four methods are presented in Figs. 5.1 and 5.2, the time evolution of the total energy of the system is plotted in Fig. 5.3. The initial conditions were [7]

$$p_1(0) = 0, \qquad q_1(0) = 1 - e, \tag{5.70}$$

and

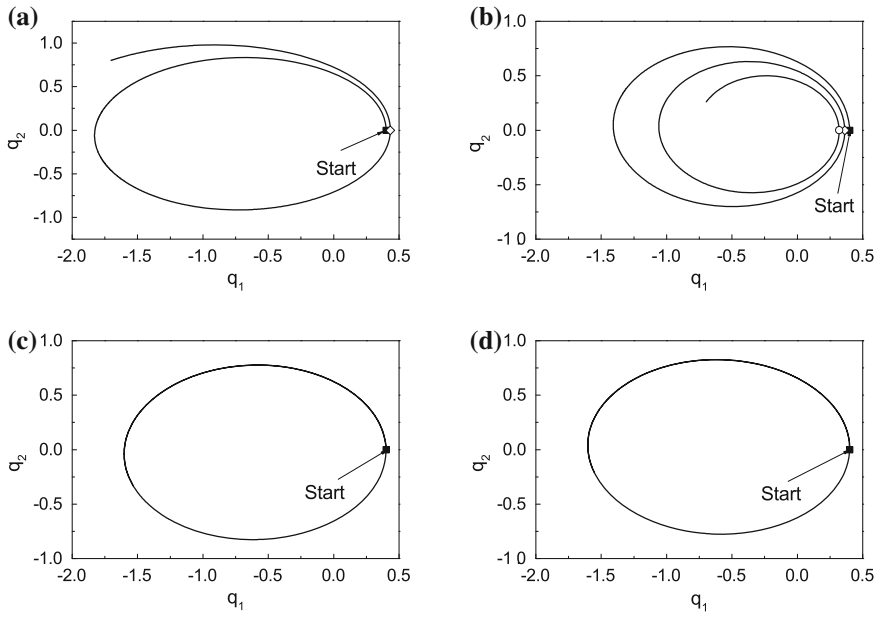


Fig. 5.1 KEPLER trajectories in position space for the initial values defined in Eqs. (5.70) and (5.71). They are indicated by a *solid square*. Solutions have been generated **(a)** by the explicit EULER method (5.65), **(b)** by the implicit EULER method (5.66), **(c)** by the symplectic EULER method (5.68), and **(d)** by the symplectic EULER method (5.69)

$$p_2(0) = \sqrt{\frac{1+e}{1-e}}, \qquad q_2(0) = 0, \tag{5.71}$$

with $e = 0.6$ which gives $H = -1/2$. Furthermore, we set $\Delta t = 0.01$ for the symplectic EULER methods and $\Delta t = 0.005$ for the forward and backward EULER methods in order to reduce the methodological error. The implicit equations were solved with help of the NEWTON method as discussed in Appendix B. The JACOBI matrix was calculated analytically, hence no methodological error enters because approximations of derivatives were unnecessary.

According to theory the q-space and p-space projections of the phase space trajectory are ellipses. Furthermore, energy and angular momentum are conserved. Thus, the numerical solutions of HAMILTON's equations of motion (5.64) should reflect these properties. Figures 5.1a, b and 5.2a, b present the results of the explicit EULER method, Eq. (5.65), and the implicit EULER method, Eq. (5.66), respectively. Obviously, the result does not agree with the theoretical expectation and the trajectories are open instead of closed. The reason for this behavior is the methodological error of the method which is accumulative and, thus, causes a violation of energy conservation. This violation becomes apparent in Fig. 5.3 where the total energy $H(t)$ is plotted versus time t. Neither the explicit EULER method (dashed line) nor the

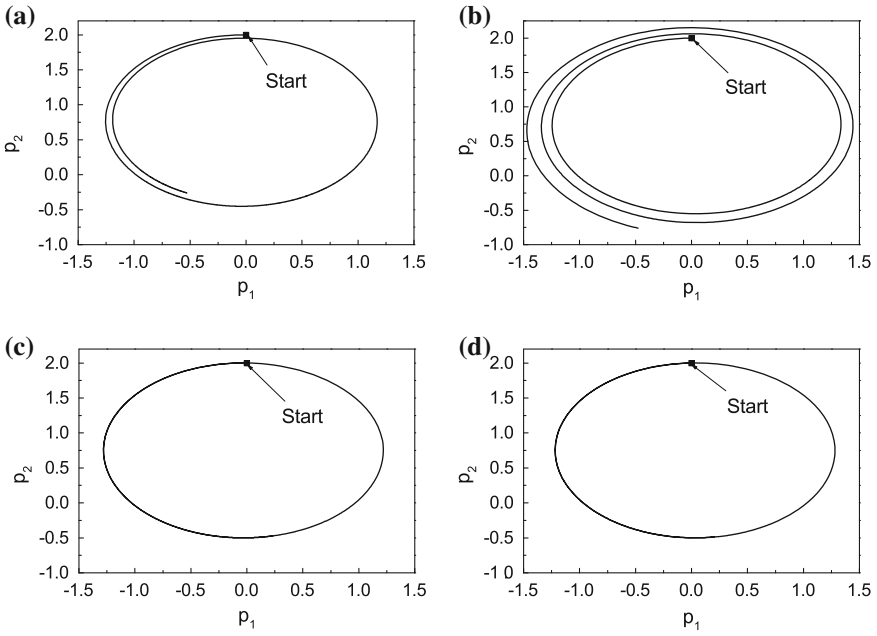


Fig. 5.2 KEPLER trajectories in momentum space for the initial values defined in Eqs. (5.70) and (5.71). They are indicated by a *solid square*. Solutions have been generated **(a)** by the explicit EULER method (5.65), **(b)** by the implicit EULER method (5.66), **(c)** by the symplectic EULER method (5.68), and **(d)** by the symplectic EULER method (5.69)

implicit EULER method (short dashed line) conform to the requirement of energy conservation. We also see step-like structures of $H(t)$. At the center of these steps an open diamond symbol and in the case of the implicit EULER method an additional open circle indicate the position in time of the perihelion of the point-mass (point of closest approach to the center of attraction). It is indicated by the same symbols in Fig. 5.1a, b. At this point the point-mass reaches its maximum velocity, the pericenter velocity, and it covers the biggest distances along its trajectory per time interval Δt. Consequently, the methodological error is biggest in this part of the trajectory which manifests itself in those steps in $H(t)$. As the point-mass moves 'faster' when the implicit EULER method is applied, again, the distances covered per time interval are greater than those covered by the point-mass in the explicit EULER method. Thus, it is not surprising that the error of the implicit EULER method is bigger as well when $H(t)$ is determined.

These results are in strong contrast to the numerical solutions of Eqs. (5.64) obtained with the help of symplectic EULER methods which are presented in Figs. 5.1c, d and 5.2c, d. The trajectories are almost perfect ellipses for both symplectic methods Eqs. (5.68) and (5.69). Moreover, the total energy $H(t)$ (solid and dashed-dotted lines in Fig. 5.3) varies very little as a function of t. Deviations from the mean value can only be observed around the perihelion which is indicated by

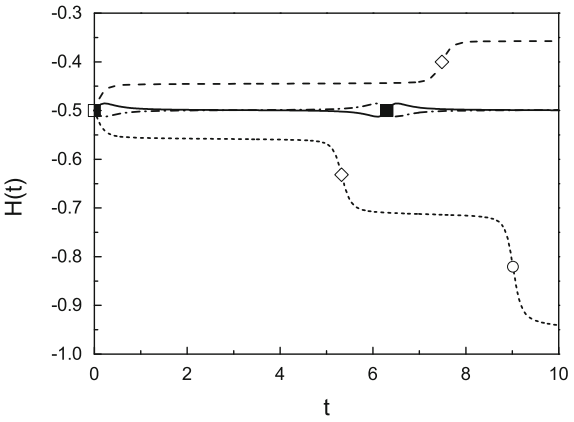


Fig. 5.3 Time evolution of the total energy H calculated with the help of the four methods discussed in the text. The initial values are given by Eqs. (5.70) and (5.71). Solutions have been generated **(a)** by the explicit EULER method (5.65) (*dashed line*), **(b)** by the implicit EULER method (5.66) (*dotted line*), **(c)** by the symplectic EULER method (5.68) (*solid line*), and **(d)** by the symplectic EULER method (5.69) (*dashed-dotted line*)

a solid square. Moreover, these deviations compensate because of the symplectic nature of the method. This proves that symplectic integrators are the appropriate technique to solve the equations of motion of Hamiltonian systems.

Summary

We concentrated on numerical methods to solve the initial value problem. The methods discussed here rely heavily on the various methods developed for numerical integration because we can always find an integral representation of this kind of ordinary differential equations. The simple integrators known from Chap. 4 were augmented by the more general CRANK-NICHOLSON method which was based on the trapezoidal rule introduced in Sect. 3.3. The simple single-step methods were improved in their methodological error by TAYLOR series methods, linear multi-step methods, and by the RUNGE-KUTTA method. The latter took intermediate points within the time interval $[t_n, t_{n+1}]$ into account. In principle, it is possible to achieve almost arbitrary accuracy with such a method. Nevertheless, all those methods had the disadvantage that because of their methodological error energy conservation was violated when applied to Hamiltonian systems. As this problem can be remedied by symplectic integrators a short introduction into this topic was provided and the most important symplectic integrators have been presented. The final discussion of KEPLER'S two-body problem elucidated the various points discussed throughout this chapter.

In contrast to the matrix (8.27) matrix (8.39) is not tridiagonal since the matrix elements $(A)_{1,N-1}$ and $(A)_{N-1,1}$ are non-zero. Nevertheless, it was possible to reduce the boundary value problem to a system of linear equations which can be solved iteratively.

8.3 Shooting Methods

Again, for illustrative purposes, we restrict the discussion to a second order boundary value problem with decoupled boundary conditions of the form (8.4). The essential idea of shooting methods is to treat the boundary value problem as an initial value problem. The resulting equations can then be solved with the help of methods discussed in Chap. 5. Of course, such an approach can, in general, not be exact. The trick is, that one modifies the initial conditions iteratively in such a way that the boundary conditions are fulfilled. Let us put this train of thoughts into a mathematical form: We rewrite the second order differential equation (8.4a) as

$$y'' = f(y, y', x), \tag{8.43}$$

which can be reduced to a set of first order differential equations as was demonstrated in Chap. 5. We note that Eq. (8.43) is not yet well posed since the initial conditions have not been defined. The boundary condition on the left hand side reads:

$$\alpha_0 y(a) + \alpha_1 y'(a) = \lambda_1. \tag{8.44}$$

We now *assume* that $y'(a) = z$, where z is some number. This gives the well posed initial value problem

$$\begin{cases} y'' = f(y, y', x), \\ y(a) = \frac{\lambda_1}{\alpha_0} - \frac{\alpha_1}{\alpha_0} z, \\ y'(a) = z, \end{cases} \tag{8.45}$$

under the assumption that $\alpha_0 \neq 0$. The solution of this problem will be written as $y(x; z)$ in order to indicate its dependence on the particular choice $y'(a) = z$. We remember, that the boundary condition at the right hand boundary reads:

$$\beta_0 y(b) + \beta_1 y'(b) = \lambda_2. \tag{8.46}$$

Let us define the function

$$F(z) = \beta_0 y(b; z) + \beta_1 y'(b; z) - \lambda_2, \tag{8.47}$$

and we observe that the solution of the equation

$$F(z) = 0, \tag{8.48}$$

gives the desired solution to the boundary value problem (8.4), because in this case the second boundary condition (8.46) is fulfilled. In practice, one tries several values of z until relation (8.48) is fulfilled. However, from a numerical point of view this method is very inefficient since usually several initial value problems have to be solved until the correct value of z is found. Nevertheless, in some cases shooting methods proved to be very useful.

For instance, shooting methods are particularly effective if a solution to an eigenvalue problem of the form

$$a(x)y''(x) + b(x)y'(x) + c(x)y(x) = \lambda y(x), \tag{8.49a}$$

in combination with homogeneous boundary conditions,

$$\alpha_0 y(a) + \alpha_1 y'(a) = 0, \tag{8.49b}$$

and

$$\beta_0 y(b) + \beta_1 y'(b) = 0. \tag{8.49c}$$

is to be found. We note that Eq. (8.49a) has the trivial solution $y(x) \equiv 0$ for all values of λ. However a non-trivial solution will only exist for particular values of λ. These particular values will be indexed by λ_n and are referred to as *eigenvalues* of Eq. (8.49a). The corresponding functions $y_n(x)$ are referred to as *eigenfunctions*. We note that the differential equation (8.49a) in combination with the boundary conditions (8.49b) and (8.49c) define a homogeneous boundary value problem. Such a problem is commonly referred to as an *eigenvalue problem*. Furthermore, we note the following property of homogeneous boundary value problems: Suppose that $y(x)$ is a solution of the boundary value problem (8.49). Then $\tilde{y}(x) = \gamma y(x)$, with $\gamma = \text{const}$ will also be a solution of (8.49). Hence, the solution of a homogeneous boundary value problem is not unique but invariant under multiplication by a constant γ. Typically, the multiplicative factor γ is fixed by some additional condition, such as a normalization condition of the form

$$\int_a^b \mathrm{d}x |y(x)|^2 = 1. \tag{8.50}$$

We now employ this property and *choose* $y(a) = 1$. Inserting this choice into (8.49b) yields

$$y'(a) = -\frac{\alpha_0}{\alpha_1}. \tag{8.51}$$

Note that for $\alpha_0 = 0$ or $\alpha_1 = 0$, we are restricted to the choices $y'(a) = 0$ and $y(a)$ is arbitrary or $y(a) = 0$ and $y'(a)$ is arbitrary, respectively. If we assume that $a(x) \neq 0$

for all $x \in [a, b]$, we can solve the initial value problem

$$\begin{cases} y''(x) &= -\frac{b(x)}{a(x)} y'(x) - \frac{c(x)-\lambda}{a(x)} y(x), \\ y(a) &= 1, \\ y'(a) &= -\frac{\alpha_0}{\alpha_1}. \end{cases} \tag{8.52}$$

The solutions are denoted by $y(x; \lambda)$ in order to emphasize that they will highly depend on the choice of the parameter λ. The strategy is to solve the initial value problem (8.52) for several values of λ and whenever one finds that

$$F(\lambda) = \beta_0 y(b; \lambda) + \beta_1 y'(b; \lambda) = 0, \tag{8.53}$$

an eigenvalue λ_n and the corresponding eigenfunction $y_n(x) = y(x; \lambda_n)$ to the eigenvalue problem (8.49) have been found.

However, this strategy is also very time consuming. The most common application of the shooting method is its combination with a very fast and accurate solution of initial value problems. This method is known as the NUMEROV method. It is applicable whenever one is confronted with a differential equation of the form

$$y''(x) + k(x) y(x) = 0, \tag{8.54}$$

in combination with homogeneous boundary conditions. Here $k(x)$ is some function. If we are particularly interested in eigenvalue problems then $k(x)$ has the form $k(x) = q(x) - \lambda$, where $q(x)$ is some function and λ is the eigenvalue [see the discussion after equation (8.49)]. For instance, consider the one-dimensional stationary SCHRÖDINGER equation,

$$\psi''(x) + \frac{2m}{\hbar^2} [E - V(x)] \psi(x) = 0, \tag{8.55}$$

where $\psi(x)$ is the wave-function, m is the mass, $\hbar$ denotes the reduced PLANCK constant, E is the energy, and $V(x)$ is some potential. In this case we identify

$$k(x) = \frac{2m}{\hbar^2} [E - V(x)]. \tag{8.56}$$

We note that Eq. (8.55) together with its boundary conditions defines an eigenvalue problem with eigenvalues E_n, the possible energies of the system. We remember from Chap. 2, Eq. (2.34), that

$$y''_j = \frac{y_{j+1} - 2y_j + y_{j-1}}{h^2} - \frac{h^2}{12} y_j^{(4)} - \cdots = -k_j y_j. \tag{8.57}$$

In the last step we made use of Eq. (8.54) and introduced $k_j \equiv k(x_j)$. Furthermore, we write the fourth derivative of $y(x)$ at point $x = x_j$ as

$$y_j^{(4)} \approx \frac{y''_{j+1} - 2y''_j + y''_{j-1}}{h^2} = \frac{-k_{j+1}y_{j+1} + 2k_j y_j - k_{j-1}y_{j-1}}{h^2}, \tag{8.58}$$

where we employed Eq. (8.54). Truncating (8.57) after the fourth order derivative $y_j^{(4)}$, inserting relation (8.58), and solving for y_{j+1} yields

$$y_{j+1} = \frac{2\left(1 - \frac{5h^2}{12}k_j\right)y_j - \left(1 + \frac{h^2}{12}k_{j-1}\right)y_{j-1}}{1 + \frac{h^2}{12}k_{j+1}}. \tag{8.59}$$

This gives a very fast algorithm to solve the differential equation (8.54) with some initial values of the form (8.52). The remaining strategy is the same as discussed above, i.e. one *screens* the parameter λ in order to find the eigenvalues λ_n and eigenfunctions $y_n(x)$. In case of the SCHRÖDINGER equation, one can screen the energy E in order to obtain the energy eigenvalues E_n which satisfy a condition of the form (8.53).

Let us conclude this section with two important remarks on the NUMEROV method before we present two illustrating examples in the next chapters. We note from Eq. (8.59) that in order to compute y_3 one already needs the function values y_1 and y_2. Usually, one obtains these values from the boundary conditions in combination with some additional condition for the problem at hand. Such an additional condition might be, for instance, the normalization of the function $y(x)$, like Eq. (8.50). Let us make a second comment. As already emphasized, one has to run the NUMEROV algorithm several times for different *trial values* of the parameter λ. In order to reduce the computational cost of the method it is in many cases advantageous to store the function values q_i, where $k_i = q_i - \lambda$, in an array, which is then regarded as an input argument of the NUMEROV algorithm.

Summary

We focused on linear boundary value problems defined on a finite interval $[a, b] \subset \mathbb{R}$. Most important for physics are second order boundary value problems with decoupled boundary conditions, i.e. the boundary conditions at the two different boundaries do not mix. The numerical treatment of the second order differential equation together with its boundary conditions concentrated either on the application of finite differences or on shooting methods. In the finite difference approach the methods developed in Chap. 2 were applied and the boundary conditions were incorporated directly. This resulted in a set of linear algebraic equations which is to be solved for each grid-point of the discretisized interval $[a, b]$. The case of periodic boundary conditions was also discussed in detail.

The shooting methods, on the other hand, try to link the decoupled boundary value problem to an initial value problem. This allowed the application of the methods discussed in Chap. 5. The idea was to start with some initial value at one of the two

boundaries, solve the differential equation numerically and to modify the initial value iteratively until it agreed with the original boundary condition within some predefined error. Such a procedure is rather time consuming. Nevertheless, shooting methods, in particular its NUMEROV variation, are very useful in the numerical solution of eigenvalue problems, as was demonstrated by the homogeneous boundary value problem of the one-dimensional stationary SCHRÖDINGER equation.

References

1. Colliatz, L.: The Numerical Treatment of Differential Equations. Springer, Berlin (1960)
2. Lapidus, L., Pinder, G.F.: Numerical Solution of Partial Differential Equations. Wiley, New York (1982)
3. Asher, U.M., Mattheij, R.M.M., Russell, R.D.: Numerical solution of boundary value problems for ordinary differential equations. Classics in Applied Mathematics, vol. 13. Cambridge University Press, Cambridge, UK (1995)
4. Press, W.H., Teukolsky, S.A., Vetterling, W.T., Flannery, B.P.: Numerical Recipes in C++, 2nd edn. Cambridge University Press, Cambridge (2002)

$$
\begin{aligned}
&= \frac{2}{n^3\pi^3}\left\{\frac{1}{2}\left[u^3 - u^2\cos(u)\sin(u)\right]_0^{n\pi} - \int_0^{n\pi} \mathrm{d}u\left[u^2 - u\cos(u)\sin(u)\right]\right\} \\
&= \frac{2}{n^3\pi^3}\left[\frac{n^3\pi^3}{2} - \frac{n^3\pi^3}{3} + \int_0^{n\pi} \mathrm{d}u u\cos(u)\sin(u)\right] \\
&= \frac{2}{n^3\pi^3}\left\{\frac{n^3\pi^3}{6} + \left[u\frac{\sin^2(u)}{2}\right]_0^{n\pi} - \frac{1}{2}\int_0^{n\pi} \mathrm{d}u \sin^2(u)\right\} \\
&= \frac{2}{n^3\pi^3}\left(\frac{n^3\pi^3}{6} - \frac{n\pi}{4}\right) \\
&= \frac{1}{3} - \frac{1}{2n^2\pi^2}\,. \qquad (10.50)
\end{aligned}
$$

Hence, the variance var (s) is determined by

$$
\left\langle (s - \langle s\rangle)^2\right\rangle = \left\langle s^2\right\rangle - \langle s\rangle^2 = \frac{1}{3} - \frac{1}{2n^2\pi^2} - \frac{1}{4} = \frac{1}{12}\left(1 - \frac{6}{n^2\pi^2}\right). \qquad (10.51)
$$

We note that the variance increases with increasing n. In the following we shall reproduce the above results with help of the shooting NUMEROV method. Furthermore, the numerical formulation will allow us to find solutions for more complex potentials.

10.3 Numerical Solution

The following discussion is based on the scaled SCHRÖDINGER equation (10.33). However, we consider a more general potential of the form

$$
v(s) = \begin{cases} \tilde{v}(s) & 0 \le s \le 1, \\ \infty & \text{elsewhere.} \end{cases} \qquad (10.52)
$$

Hence, the boundary value problem (10.34) takes on the form

$$
\begin{cases} -\frac{1}{2}\frac{\mathrm{d}^2}{\mathrm{d}s^2}\varphi_n(s) + \tilde{v}(s)\varphi_n(s) = \varepsilon_n\varphi_n(s) & s \in [0,1], \\ \varphi_n(0) = 0, \\ \varphi_n(1) = 0. \end{cases} \qquad (10.53)
$$

Moreover, our numerical treatment will be based on shooting methods, discussed in Sect. 8.3. Therefore, we rewrite the second order differential equation in Eq. (10.53) in a form which corresponds to Eq. (8.54), namely:

$$\varphi_n''(s) + 2\left[\varepsilon_n - \tilde{v}(s)\right]\varphi_n(s) = 0. \tag{10.54}$$

The interval $[0, 1]$ is discretized using $N + 1$ grid-points $s_\ell = \ell/N$, $\ell = 0, 1, 2, \ldots, N$ ($h = 1/N$) and we denote with $\varphi_n(s_\ell)$ and $\tilde{v}(s_\ell) \equiv v_\ell$ the values of $\varphi_n(s)$ and $\tilde{v}(s)$ at those grid-points. This allows to rewrite Eq. (8.59) as

$$\varphi_n(s_{\ell+1}) = \frac{2\left[1 - \frac{5}{6N^2}\left(\varepsilon_n - \tilde{v}_\ell\right)\right]\varphi_n(s_\ell) - \left[1 + \frac{1}{6N^2}\left(\varepsilon_n - \tilde{v}_{\ell-1}\right)\right]\varphi_n(s_{\ell-1})}{1 + \frac{1}{6N^2}\left(\varepsilon_n - \tilde{v}_{\ell+1}\right)}. \tag{10.55}$$

Again, we will denote solutions of this equation by $\varphi_n(s_\ell; \varepsilon)$ to emphasize their dependence on the eigenvalue ε.

We use the initial conditions $\varphi_n(s_0) = 0$ and $\varphi_n'(s_0) = 1$, which is always possible, since (10.53) is a homogeneous boundary value problem. This gives

$$\varphi_n'(s_0) \approx \frac{\varphi_n(s_1) - \varphi_n(s_{-1})}{2h} = 1 \quad \Rightarrow \quad \varphi_n(s_1) = \frac{2}{N}\,. \tag{10.56}$$

The normalization of the wave-function (10.30) is then approximated with the help of the forward rectangular rule (3.9):

$$\int_0^1 \mathrm{d}s |\varphi_n(s)|^2 \approx h \sum_{\ell=0}^{N} [\varphi_n(s_\ell)]^2 \stackrel{!}{=} 1. \tag{10.57}$$

Consistently, we approximate the expectation value $\langle s \rangle$ via

Table 10.1 Comparison between analytic and numerical eigenenergies for the particle in a box for $N = 100$

n	ε_n-analytic	ε_n-numeric
1	4.934802	4.934802
2	19.739209	19.739208
3	44.413219	44.413205
4	78.956835	78.956753
5	123.370055	123.369742
6	177.652879	177.651943
7	241.805308	241.802947
8	315.827341	315.822077
9	399.718978	399.708300
10	493.480220	493.460113

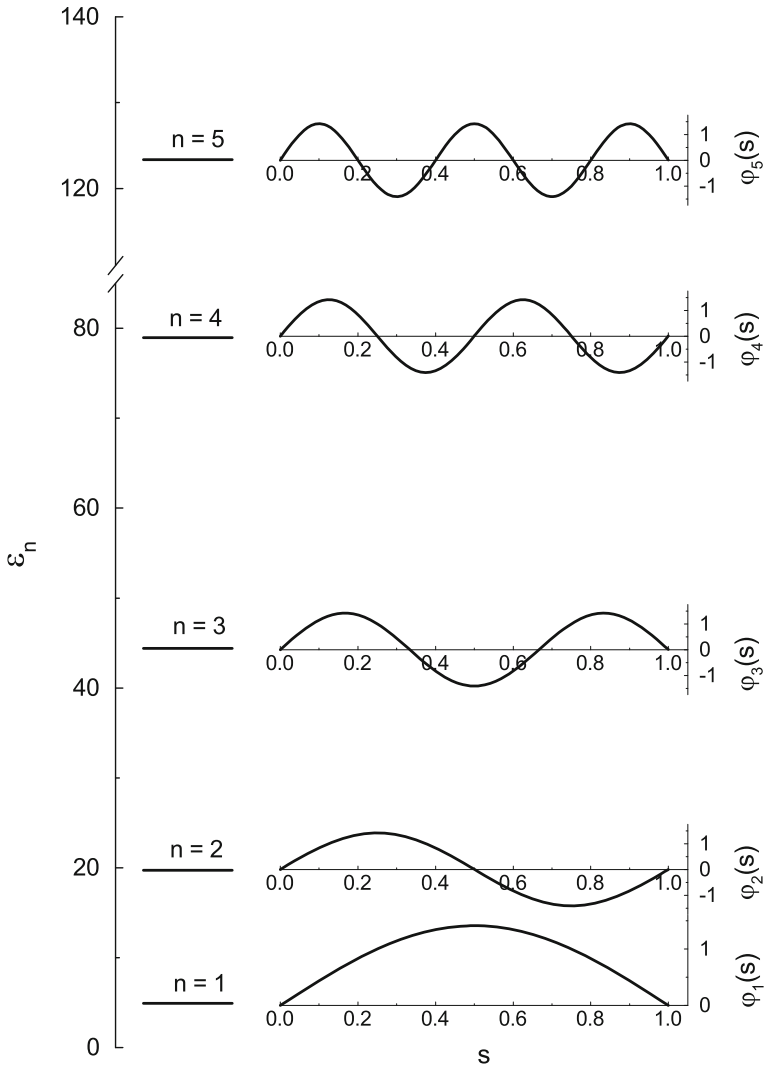


Fig. 10.1 The first five numerically determined eigenvalues ε_n of Table 10.1 are presented has horizontal lines (*left hand scale*). Aligned with these eigenvalues are the corresponding eigenfunctions $\varphi_n(s)$ versus s for $N = 100$ (*right hand scales*)

$$\int_0^1 \mathrm{d}s\, s|\varphi_n(s)|^2 \approx h \sum_{\ell=0}^{N} [\varphi_n(s_\ell)]^2\, s_\ell \ . \tag{10.58}$$

The NUMEROV shooting algorithm is then defined by the following steps:

1. Choose two trial energies ε_a and ε_b and define the required accuracy η.
2. Calculate $\varphi(s_N; \varepsilon_a) \equiv \varphi_a$ and $\varphi(s_N; \varepsilon_b) \equiv \varphi_b$ using Eq. (10.55).

3. If $\varphi_a \varphi_b > 0$, choose new values for ε_a or ε_b and go to step 1.
4. If $\varphi_a \varphi_b < 0$, calculate $\varepsilon_c = (\varepsilon_a + \varepsilon_b)/2$ and determine $\varphi(s_N; \varepsilon_c) \equiv \varphi_c$ using Eq. (10.55).
5. If $\varphi_a \varphi_c < 0$, set $\varepsilon_b = \varepsilon_c$ and go to step 4.
6. If $\varphi_c \varphi_b < 0$, set $\varepsilon_a = \varepsilon_c$ and go to step 4.
7. Terminate the loop when $|\varepsilon_a - \varepsilon_b| < \eta$.

These steps have been carried out for 100 grid-points, a potential $\tilde{v} = 0$, and a required accuracy of $\eta = 10^{-10}$. The first ten eigenenergies are given in Table 10.1 and are compared with the analytic results (10.40).

In addition, Fig. 10.1 presents the first five eigenvalues ε_n (right hand scale) as horizontal straight lines. Aligned with these eigenvalues we find on the right hand side of this figure the corresponding normalized eigenfunctions calculated using $N = 100$ grid-points. The agreement with the analytic result of Eq. (10.41) is excellent.

10.4 Another Case

Here we discuss briefly some results achieved with the help of NUMEROV's shooting algorithm. In particular, we discuss the particle in the box for three different potentials $\tilde{v}(s)$ [see Eq. (10.52)],

$$\tilde{v}_1(s) = 50\cos(\pi s), \quad \tilde{v}_2(s) = 50\exp\left[-\frac{\left(s-\frac{1}{2}\right)^2}{0.08}\right], \quad \tilde{v}_3(s) = 50s. \qquad (10.59)$$

The potentials are illustrated in Fig. 10.2. All calculations were carried out with $N = 100$ grid-points and an accuracy $\eta = 10^{-10}$. The first five eigenenergies ε_n are shown in Figs. 10.3, 10.4 and 10.5, respectively, as horizontal lines (left hand scale).

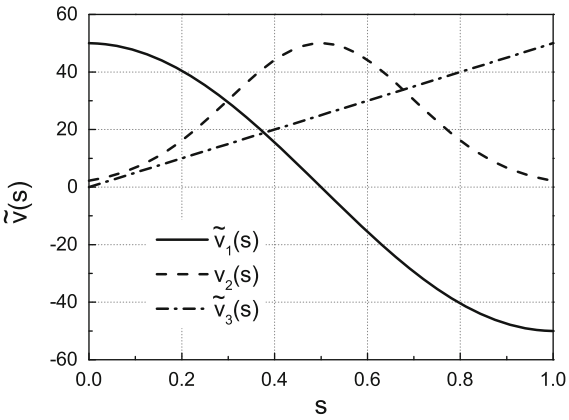


Fig. 10.2 The three different potentials $\tilde{v}(s)$ introduced by Eq. (10.59)

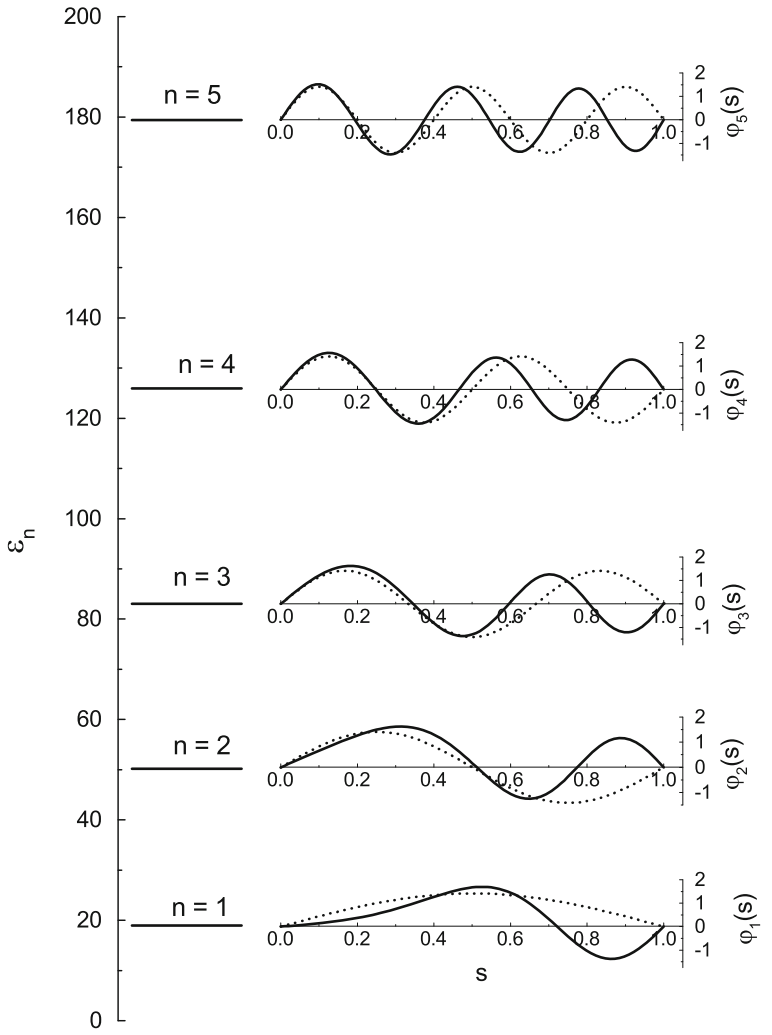


Fig. 10.3 Numerically determined eigenvalues ε_n (*left hand scale*) and eigenfunctions $\varphi_n(s)$ versus s (*right hand scales*) for the potential $\tilde{v}_1(s)$. The first five eigenvalues are presented as straight *horizontal lines*. Aligned with these lines the eigenfunctions are shown on the *right hand side* of this figure. The *dotted lines* indicate the eigenfunctions of the particle in the box with $\tilde{v}(s) = 0$ (see Fig. 10.1)

The numerically determined normalized eigenfunctions $\varphi_n(s)$ versus s (solid lines) are presented on the right hand side of these figures and are aligned with their respective eigenvalues. They are also compared with the eigenfunctions (dotted lines) of the particle in a box, i.e. $\tilde{v}(s) = 0$. In all cases the eigenfunctions reflect the symmetry of the various potentials $\tilde{v}(s)$ which becomes particularly transparent in Fig. 10.3 for the potential $\tilde{v}_1(s)$. The eigenfunctions develop an additional node in

Chapter 14
A Brief Introduction to Monte-Carlo Methods

14.1 Introduction

It is the aim of this chapter to give a brief introduction to Monte-Carlo methods in general, and to Monte-Carlo integration as well as to the METROPOLIS-HASTINGS algorithm in particular. A detailed discussion of the fundamental concepts involved is postponed to Chap. 16. The introduction given here is not supposed to be self-contained. Actually, we shall introduce concepts without referring to the origin of the relations employed.

The notion of Monte-Carlo methods, Monte-Carlo algorithms or Monte-Carlo techniques is not well defined. In particular, the term *Monte-Carlo* summarizes a wide field of methods, which are based on the sampling of random numbers. In general, the advantage of Monte-Carlo algorithms can be found in their computational strength. In many cases it is simply not feasible to employ deterministic methods due to their very high computational cost. However, in many cases the use of methods based on random sampling is also motivated by the nature of the processes to be described. We mentioned in the previous chapter as a typical example the radioactive decay of some nucleus. This process is believed to be purely stochastic in nature.

The development of Monte-Carlo techniques was initialized in the 1940s by VON NEUMANN, ULAM and METROPOLIS, who actually also coined the term *Monte-Carlo methods*. In particular, ULAM's uncle used to gamble (and loose his money) in a casino named *Monte-Carlo casino*, which they apparently regarded as their inspiration.

One of the earliest and most impressive illustration of the principle of Monte-Carlo techniques in general, and of Monte-Carlo integration in particular is the Monte-Carlo approximation of π. The discussion which follows now includes the essential ideas of Monte-Carlo integration.

We regard the unit square characterized by the corner points (0, 0), (0, 1), (1, 0), and (1, 1). The area A_s of this square is one. We insert a quarter-circle of radius $r = 1$ which, consequently, possesses the area $A_c = \pi/4$. Suppose, we are throwing darts on this unit square in such a way that the impact points are uniformly distributed; then the probability P that a certain dart becomes stuck within the interior of the

B. A. Stickler and E. Schachinger, *Basic Concepts in Computational Physics*,
DOI: 10.1007/978-3-319-02435-6_14, © Springer International Publishing Switzerland 2014

Table 14.1 Approximate values $\pi_a^{(i)}$ obtained with the method discussed in the text

N	$\pi_a^{(1)}$	$\lvert\pi_a^{(1)} - \pi\rvert$	$\pi_a^{(2)}$	$\lvert\pi_a^{(2)} - \pi\rvert$
10	2.8000	0.34159	2.8000	0.34159
10^2	2.9200	0.22159	3.1600	0.01841
10^3	3.1600	0.01841	3.1840	0.04241
10^4	3.1304	0.01119	3.1868	0.04521
10^5	3.1358	0.00579	3.1875	0.04589
10^6	3.1393	0.00229	3.1875	0.04599
10^7	3.1413	0.00028	3.1875	0.04599

The linear congruential generators are initialized by the following parameters: generator (1): $a = 7^5$, $c = 0$, $m = 2^{31} - 1$, and $x_0 = 281$ (PARK-MILLER) and generator (2): $a = 7^5$, $c = 0$, $m = 2^{10}$, and $x_0 = 281$. We also give the absolute errors $\lvert\pi_a^{(i)} - \pi\rvert$

quarter-circle is given by

$$P = \frac{A_c}{A_s} = A_c = \frac{\pi}{4} = 0.785398\ldots. \tag{14.1}$$

From a probabilistic point of view, we have after N throws of which n hit the interior of the quarter-circle the probability:

$$P = \lim_{N\to\infty} \frac{n}{N}. \tag{14.2}$$

The strategy is clear: we draw random numbers x_i, y_i from the interval $[0, 1]$. These are the intersection points of the darts. Repeating this *experiment* several times and counting the number of hits n within the quarter-circle allows us to approximate π via

$$P = \frac{\pi}{4} \approx \frac{n}{N}. \tag{14.3}$$

The resulting approximation of π will be strongly influenced by the number of experiments N as well as by the performance of the random number generator used. Table 14.1 lists computed approximations of π for different numbers of experiments N as they were obtained with the help of a linear congruential generator. Linear congruential generators have been introduced and discussed in Sect. 12.2. The parameters used to initialize the generators are given in the caption of the table. Furthermore, Fig. 14.1 illustrates the result after $N = 10^3$ experiments for both generators.

14.2 Monte-Carlo Integration

We generalize the ideas formulated above: we consider a function $f(x) \geq 0$ for $x \in [a, b] \subset \mathbb{R}$ where the area of interest is

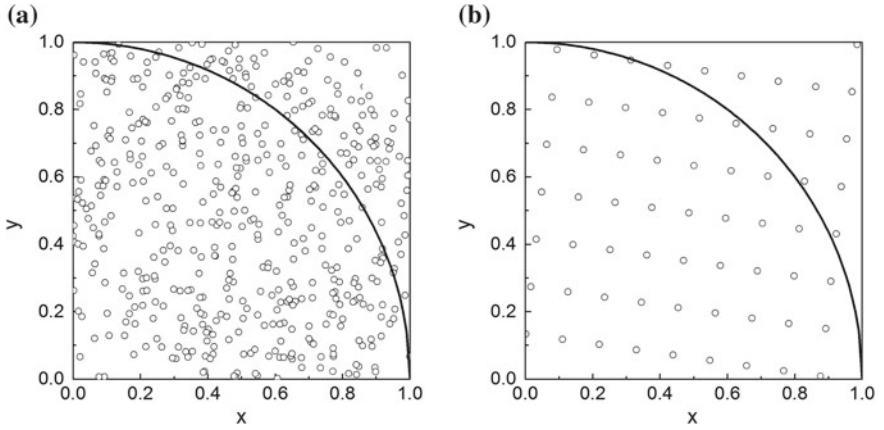


Fig. 14.1 $N = 10^3$ uniformly distributed random numbers within the unit-square. Frame (**a**) gives the results for generator (1) while frame (**b**) is for generator (2). The number of elements within the quarter-circle indicated by the solid line determines the value of $\pi_a^{(i)}$. The inferior result of the approximation obtained with generator (2) [frame (**b**)] originates in correlations between the x and y coordinates

$$A = \int_a^b \mathrm{d}x \; f(x). \tag{14.4}$$

We denote

$$\xi = \max_{x \in [a,b]} f(x), \tag{14.5}$$

and obtain using the above example

$$A = A_s \lim_{N \to \infty} \frac{n}{N}, \tag{14.6}$$

where n is the number of random points under the curve indicated schematically in Fig. 14.2. The area A_s is given by

$$A_s = (b - a)\xi, \tag{14.7}$$

and the random numbers $r_i = (x_i, y_i)$ are uniformly distributed within the intervals $x_i \in [a, b]$ and $y_i \in [0, \xi]$. This method is referred to as *hit and miss integration*.

Another way to perform a Monte-Carlo integration is the so called *mean-value integration* . It is essentially based on the mean value theorem of calculus which we already employed in our discussion of quadrature in Chap. 3. We will restate the theorem here for the sake of a more transparent illustration. The mean-value theorem states that if $f(x)$ is a continuous function for $x \in [a, b]$ then there exists a $z \in (a, b)$ such that

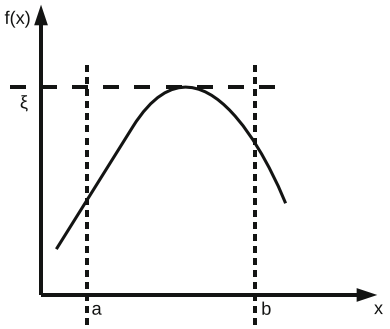


Fig. 14.2 Schematic illustration of the Monte-Carlo integration technique

$$\int_a^b \mathrm{d}x\ f(x) = f(z)(b-a). \tag{14.8}$$

The function value $f(z) = \langle f \rangle$ is referred to as the *expectation value* or *mean value* of $f(x)$. Now we know from probability theory [1, 2], that the expectation value can be approximated by the arithmetic mean $\overline{f}$, with the error given by the standard error, i.e.

$$\frac{1}{b-a}\int_a^b \mathrm{d}x'\ f(x') = \overline{f} \pm \sqrt{\frac{\overline{f^2} - \overline{f}^2}{N}}. \tag{14.9}$$

The arithmetic mean $\overline{f}$, on the other hand, is given by

$$\overline{f} = \frac{1}{N}\sum_{i=1}^{N} f(x_i), \tag{14.10}$$

and consequently

$$\overline{f^2} = \frac{1}{N}\sum_{i=1}^{N} f^2(x_i). \tag{14.11}$$

Note that here the variables x_i are assumed to be uniformly distributed random numbers within the interval $[a, b]$. We will immediately discuss this result in more detail. However, first of all we note from the law of large numbers, Appendix D, that this approach is exact in the limit $N \to \infty$, i.e.

$$\frac{1}{b-a}\int_a^b \mathrm{d}x'\ f(x') = \lim_{N\to\infty} \frac{1}{N}\sum_{i=1}^{N} f(x_i). \tag{14.12}$$

Let us consider the more general case: We want to evaluate the expectation value

$$\langle f \rangle = \int \mathrm{d}x \; f(x) p(x), \tag{14.13}$$

where $x \in \mathbb{R}^d$ and $p(x)$ is a pdf. A typical example is the calculation of the thermal expectation value in statistical physics where the pdf $p(x)$ is given by the normalized BOLTZMANN distribution

$$p(x) = \frac{1}{Z} \exp\left[-\frac{E(x)}{k_B T}\right]. \tag{14.14}$$

Here $E(x)$ denotes the energy as a function of the parameter $x \in \mathbb{R}^d$, k_B stands for BOLTZMANN's constant, T is the temperature, and the normalization factor Z is referred to as the canonical partition function.

Equation (14.13) may be rewritten as

$$\langle f \rangle = \int \mathrm{d}x \; f(x) p(x) = \int \mathrm{d}f \; f q(f), \tag{14.15}$$

where we introduced the probability density $q(f)$ of f via

$$q(f) = \int \mathrm{d}x \; \delta[f - f(x)] \, p(x). \tag{14.16}$$

with $\delta(\cdot)$ DIRAC's δ-distribution. Let us briefly explain how we arrived at this definition. Let the cdf $P(x)$ be defined by[1]

$$P(x) = \Pr(X \leq x) = \int_{-\infty}^{x} \mathrm{d}x \; p(x). \tag{14.17}$$

We define in a similar fashion the cdf $Q(f)$ via

$$Q(f) = \Pr(F \leq f) = \Pr[f(X) \leq f]. \tag{14.18}$$

Note that we distinguish between the function $f(X)$ of the random variable X—which in turn follows the pdf $p(X)$—and the particular function value $f \in \mathbb{R}$. Furthermore, the probability $\Pr[f(X) \leq f]$ can be rewritten as

$$\Pr[f(X) \leq f] = \sum_n \Pr(a_n \leq X \leq b_n), \tag{14.19}$$

where the values $a_n < b_n$ are the ordered intersection points $a_1 < b_1 < a_2 < b_2 < \cdots < a_N < b_N$ chosen in such a way that

[1] Please note that according to the conventions established in Appendix D capital letters denote random variables.

$$f(a_n) = f(b_n) = f, \quad \text{and} \quad f[x \in (a_n, b_n)] < f. \tag{14.20}$$

It is a matter of the particular form of $f(x)$ whether or not the boundary points have to be included. Equation (14.19) can be rewritten:

$$\Pr(a_n \leq X \leq b_n) = P(b_n) - P(a_n) = \int_{a_n}^{b_n} \mathrm{d}x \; p(x). \tag{14.21}$$

The pdf $q(f)$ is related to the cdf $Q(f)$ via

$$q(f) = \frac{\mathrm{d}}{\mathrm{d}f} Q(f), \tag{14.22}$$

and we obtain

$$\begin{aligned} q(f) &= \sum_n \frac{\mathrm{d}}{\mathrm{d}f} \Pr(a_n \leq X \leq b_n) \\ &= \sum_n \frac{\mathrm{d}}{\mathrm{d}f} \int_{a_n}^{b_n} \mathrm{d}x \; p(x) \\ &= \sum_n \left[\frac{\mathrm{d}b_n}{\mathrm{d}f} p(b_n) - \frac{\mathrm{d}a_n}{\mathrm{d}f} p(a_n) \right] \\ &= \sum_n \left[\left(\frac{\mathrm{d}f(x)}{\mathrm{d}x} \right)^{-1} p(x) \Big|_{x=b_n} - \left(\frac{\mathrm{d}f(x)}{\mathrm{d}x} \right)^{-1} p(x) \Big|_{x=a_n} \right]. \end{aligned} \tag{14.23}$$

However, from Eq. (14.20) we know that

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x}\bigg|_{x=b_n} \overset{!}{>} 0 \quad \text{and} \quad \frac{\mathrm{d}f(x)}{\mathrm{d}x}\bigg|_{x=a_n} \overset{!}{<} 0. \tag{14.24}$$

We introduce the intersection points x_k where $x_1 < x_2 < \cdots < x_K$ and $K = 2N$—if the boundary points are not included—for which $f(x_1) = f(x_2) = \cdots = f(x_K) = f$. Hence, Eq. (14.23) may be rewritten as

$$\begin{aligned} q(f) &= \sum_k \left| \frac{\mathrm{d}f\ (x)}{\mathrm{d}x} \right|^{-1} \Bigg|_{x=x_k} p(x_k) \\ &= \sum_k \frac{p(x_k)}{|f'(x_k)|}. \end{aligned} \tag{14.25}$$

The DIRAC δ-distribution of an arbitrary function $g(y)$ can be expressed as

$$\delta[g(y)] = \sum_i \frac{\delta(y - y_i)}{|g'(y_i)|}, \tag{14.26}$$

where the y_i are the zeros of $g(y)$, i.e. $g(y_i) = 0$. Hence, we arrive at the final result and find for Eq. (14.23):[2]

$$q(f) = \int \mathrm{d}x \; \delta\left[f - f(x)\right] p(x). \tag{14.27}$$

Furthermore, we note that

$$\int \mathrm{d}f \; q(f) = \int \mathrm{d}f \int \mathrm{d}x \; \delta\left[f - f(x)\right] p(x) = \int \mathrm{d}x \; p(x) = 1. \tag{14.28}$$

As a result, the variance of f, var (f) can be expressed as

$$\mathrm{var}\,(f) = \int \mathrm{d}x \; \left[f(x) - \langle f \rangle\right]^2 p(x) = \int \mathrm{d}f \; \left[f - \langle f \rangle\right]^2 q(f). \tag{14.29}$$

Let us define in a next step the arithmetic mean

$$\mathscr{F} = \frac{1}{N} \sum_{i=1}^{N} f(x_i) = \frac{1}{N} \sum_{i=1}^{N} f_i, \tag{14.30}$$

calculated with the help of N random numbers. Hence, we have

$$\langle \mathscr{F} \rangle = \langle f \rangle, \tag{14.31}$$

and

$$\mathrm{var}\,(\mathscr{F}) = \frac{\mathrm{var}\,(f)}{N}, \tag{14.32}$$

according to Appendix D. It follows from the central limit theorem, that for large values of N, the pdf of $\mathscr{F}$, $p(\mathscr{F})$ converges to a normal distribution $\mathscr{N}$ with $\langle \mathscr{F} \rangle$ and var $(\mathscr{F})$, i.e.

---

[2] We give an example. Suppose $f(x) = \exp(x)$. Then we deduce that

$$\delta\left[f - \exp(x)\right] = \frac{\delta(x - \ln f)}{f},$$

and, consequently,

$$q(f) = \frac{p(\ln f)}{f}.$$

$$p(\mathscr{F}) \approx \mathscr{N}\left(\mathscr{F}\,\middle|\,\langle f\rangle\,,\,\frac{\mathrm{var}\,(f)}{N}\right). \tag{14.33}$$

This property can now be employed to estimate $\langle f\rangle$ according to

$$\langle f\rangle = \mathscr{F} \pm \sqrt{\frac{\mathrm{var}\,(f)}{N}} = \frac{1}{N}\sum_{i=1}^{N} f(x_i) \pm \sqrt{\frac{\mathrm{var}\,(f)}{N}}, \tag{14.34}$$

with random numbers x_i sampled from the pdf $p(x)$. This method is the most prominent formulation of Monte-Carlo integration.

We shall briefly discuss some properties of this method. We deduce from Eq. (14.34) that the error scales like $N^{-\frac{1}{2}}$. In contrast to the integration methods we discussed in Chap. 3, N is no longer the number of grid-points but the number of random numbers sampled.[3] In principle, the error scaling is worse than in the case of classical integrators. For instance, in the case of the central rectangular rule (Sect. 3.2) we had an error scaling of N^{-2} when summed over the whole interval. However, we obtained this result for the one-dimensional case, in higher dimensions we will certainly need much more grid-points. On the other hand, in Eq. (14.34) N corresponds the number of d-dimensional random numbers x. Hence, Monte-Carlo integration can be of advantage whenever one has to deal with complicated, high dimensional integrals. In contrast, restricted to one dimension it is in most cases not an improvement of the methods discussed already.

Monte-Carlo integration can also be of advantage whenever the integrand $f(x)$ is not well behaved, i.e. if one would need a very fine grid in order to compute a reasonable estimate of the true value of the integral. In such cases Monte-Carlo integration offers a very convenient approach due to its conceptual simplicity.

It is certainly a drawback of Monte-Carlo integration in its formulation (14.34), that the error is also proportional to $\sqrt{\mathrm{var}\,(f)}$, which is a yet unknown quantity. One has to approximate it with an adequate estimator, for instance with the help of the sampling variance. Moreover, if the variance var (f) diverges, the central limit theorem does not hold and the procedure (14.34) is no longer justified and will fail for sure.

Closely related to the problem of how to determine var (f), is the question of how many random numbers should be drawn. In most cases an iterative approach is the most promising strategy. In a first step N random numbers are drawn and the integral is computed using Eq. (14.34). Then another set of N random numbers is sampled and Eq. (14.34) is reevaluated now using all $2N$ random numbers. If the change in the resulting estimate of the integral is less than some given tolerance ε, the loop is terminated otherwise another set of N random numbers is added.

We mention that this form of Monte-Carlo integration can be improved particularly by sampling only from points which dominantly contribute to the integral. This

[3] Nevertheless, there is certainly some conceptual similarity between grid-points and random numbers within this context.

method is referred to as *importance sampling* and will be discussed in more detail later on.

14.3 The METROPOLIS Algorithm: An Introduction

Within this section we shall briefly discuss the METROPOLIS algorithm as a more sophisticated method to produce random numbers from given distributions. In fact, the METROPOLIS algorithm is a special form of the rejection method (Sect. 13.3). The aim of this particular section is to introduce the algorithm on a very basic level in order to allow a first view at an interesting model out of statistical physics, namely the ISING model which will be discussed in Chap. 15. A more detailed discussion of the METROPOLIS algorithm is postponed to Sect. 16.4.

The METROPOLIS algorithm is particularly useful to treat problems in statistical physics where thermodynamic expectation values of some observable O are the interesting objects. They are defined as

$$\langle O \rangle = \int \mathrm{d}x \; O(x) q(x), \tag{14.35}$$

where x is a set of parameters and $q(x)$ is the BOLTZMANN distribution (14.14). The set of parameters x could be, for instance, the position- and momentum-space coordinates of N different particles. In most cases x is a high dimensional object, which makes classical numerical integration (Chap. 3) cumbersome. Instead Monte-Carlo integration is employed and the integral (14.35) is approximated with the help of Eq. (14.34)

$$\langle O \rangle \approx \frac{1}{N} \sum_{i=1}^{N} O(x_i) \pm \sqrt{\frac{\mathrm{var}\,(O)}{N}}, \tag{14.36}$$

where the uncorrelated random numbers $x_i, i = 1, 2, \ldots, N$ are drawn according to the pdf $q(x)$ Eq. (14.14). We recognize immediately the problem: we need to know the exact functional form of $q(x)$ if we want to apply either the inverse transformation method or the rejection method discussed in Chap. 13. However, the partition function Z itself is determined by an integral which can be approximated using Eq. (14.36). We set

$$q(x) = \frac{p(x)}{Z}, \tag{14.37}$$

and

$$Z = \int \mathrm{d}x \; p(x) \tag{14.38}$$

follows from the normalization of $q(x)$. The METROPOLIS algorithm was designed to avoid precisely this problem. We concentrate on a pdf which is of the form (14.37), but $q(x)$ must not necessarily be of the form (14.14), i.e. $p(x)$ is arbitrary but it

At $x = 0$, $p = 0$

$$\frac{dy}{dx} = \frac{1}{h}\left[\Delta y_0 + \frac{\Delta^2 y_0}{2!} + \frac{\Delta^3 y_0}{3!} + \dots\right] \tag{7.17}$$

To verify these equations again we use data generated from $y = x^3$. The program *fd-derivative-test.py* uses the results from the forward difference table and calculates the value of the derivative at $x = 1.5$ by using equation 7.16. The result is cross checked against $3x^2$, the derivative of x^3.

```
x = [0,1,2,3]
y = [0,1,8,27]
a = [0,1,3,1]   #coefficients from FD table

h = x[1] - x[0]
nx = 1.5   # new point
p = (nx-x[0])/h
dny = a[1] + (2*p-1)*a[2] + (3*p**2-6*p+2)*a[3]
print  (dny, 3*nx**2)
```

7.4.1 Numpy gradient function

The numpy module provides functions to differentiate data sets available in numerical form. Program **diff-data.py** loads two columns of data, x and y, from a file, calculates the first and second derivatives and plots them. The program is shown below.

```
import numpy as np
import matplotlib.pyplot as plt
x,y = loadtxt("diff-data.txt", delimiter=' ')   # read 2 column data
dx = x[1] - x[0]
dy = np.diff(y)/dx
d2y = np.diff(dy)/dx
plt.subplot(1,3,1)
plt.plot(x,y)
plt.subplot(1,3,2)
plt.plot(x[:-1],dy)
plt.subplot(1,3,3)
plt.axis([0,10,0,20])
plt.plot(x[:-2],d2y)
plt.show()
```

Output of the program in shown in figure7.3.The input data looks like a parabola, first derivative has constant slope and the second derivative is a constant. The data is generated by the following lines of code.

```
t = linspace(0,10,11)
s = 5 * t**2    #S = ut + 1/2 at^2
savetxt('diff-data.txt', array([t,s]), delimiter = ' ')
```

7.5 First Order Ordinary Differential Equations

Differential equations are one of the most important mathematical tools used in producing models for physical and biological processes. In this section, we will discuss the numerical methods

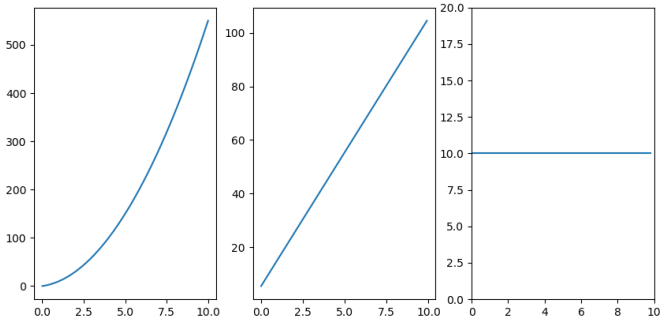


Figure 7.3: Numpy gradient function demonstration

for solving the initial value problem for first-order ordinary differential equations. Consider the equation,

$$\frac{dy}{dx} = f(x, y); \quad y(x_0) = y_0 \tag{7.18}$$

where the derivative of the function $f(x, y)$ is known everywhere, and the value of the function at some value of $x = x_0$ also is known. The objective is to find out the value of the function for other values of x. The underlying idea of any routine for solving the initial value problem is to rewrite the dy and dx as finite steps $\triangle y$ and $\triangle x$, and multiply the equations by $\triangle x$, and use the Taylor series as explained in the previous section.

Implementation of this procedure as such results in the Euler's method. Euler method is very simple and demonstrates the concept of using Taylor series. The method is not accurate due to the following reasons. The derivative at the starting point is used for computing the function during the interval Δx, that may not be true. The higher order derivatives are ignored. Later we will explore some methods that remedy this drawbacks.

7.5.1 Euler method

The equations of Euler's method can be obtained as follows. By the definition of derivative,

$$y^{'}(x_n, y_n) = \lim_{h \to 0} \frac{y(x_n + h) - y(x_n)}{h} \tag{7.19}$$

For sufficiently small values of h , we can write,

$$y(x_n + h) = y(x_n, y_n) + hy^{'}(x_n) \tag{7.20}$$

The above equations implies that, if the value of the function $y(x)$ is known to be y_n at the point x_n, its value at a nearby point x_{n+1} is given by $y_n + h \times y^{'}$. The program *euler.py* calculates the value of sine function using its derivative, ie. the cosine function. We start from $x = 0$, where $\sin(x) = 0$ and compute the subsequent values using the derivative, $cos(x)$, and compare the result with the actual sine function. The requirement is to calculate the value of a function during a given interval. The values of the function and it's derivative is given at the starting point. We will define a numpy array of the values of the independent variable x during the interval with a spacing of Δx between two adjacent points. An empty array of the same size is created to fill in the computed values of the function.

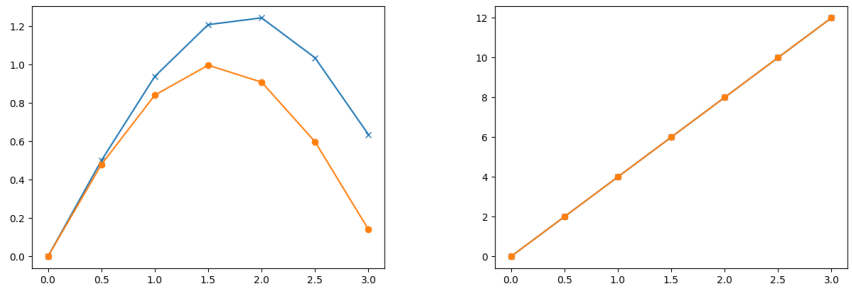


Figure 7.4: Outputs of (a)euler-sine.py.(b)modified for line

Example **euler-sine.py**

```
import numpy as np
import matplotlib.pyplot as plt
def f1(x,y):
  return np.cos(x)
def euler(x, y, fxy, h):
    return y + h * fxy(x,y)   # Euler method

dx = .5   # step size
xmin = 0  # initial value
xmax = np.pi # calculate up to this only
xa = np.arange(xmin, xmax, dx)
N =len(xa)
ya = np.empty(N) # numpy array to store results
ya[0] = 0.0      # given initial value

for i in range(N-1):
    ya[i+1] = euler(xa[i], ya[i], f1, dx)
plt.plot(xa, ya, 'x-')
plt.plot(xa, np.sin(xa), 'o-')
plt.show()
```

The output of *euler-sine.py* is shown in figure 7.4. A large step size is chosen to highlight the error. Modify the code to reduce dx to see the result. To understand the source of error, modify the function $f1$ to return a number, like *return* 4. This represents a straight line with a slope of 4. The results are shown in figure .

7.5.2 Second order Runge-Kutta method

The formula 7.19 used by Euler method which advances a solution from x_n to x_{n+1} is not symmetric, it advances the solution through an interval h, but uses derivative information at the beginning of that interval. Better results are obtained if we take *trial* step to the midpoint of the interval and use the value of both x and y at that midpoint to compute the *real* step across the whole interval. This is called the second-order Runge-Kutta method or the midpoint method. The only changes required in the code are replacing the function 'euler' by 'rk2' as shown below.

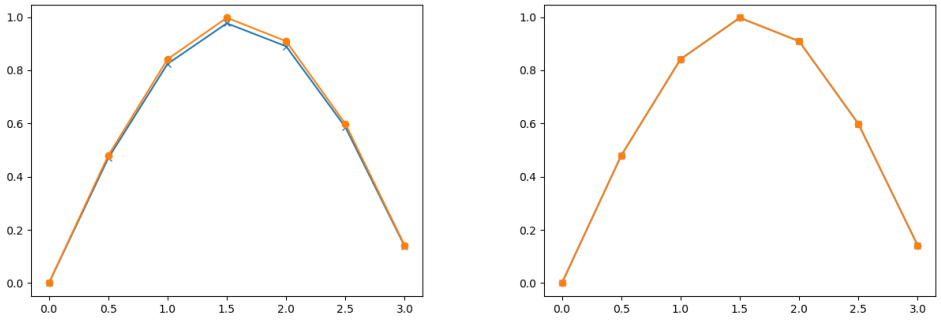


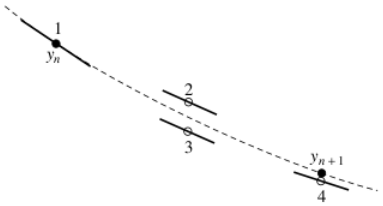
Figure 7.5: Outputs of (a) rk2.py (b)rk4.py

```
def rk2(x, y, fxy, h):
    k1 = fxy(x, y)
    k2 = fxy(x + h, h*k1)
    return y + h * ( k1/2 + k2/2)
```

The result is shown in figure 7.5(a).

7.5.3 Fourth order Runge-Kutta method

The fourth order Runge-Kutta method is the most popular one and is commonly referred as the Runge-Kutta method. In each step the derivative is evaluated four times, once at the initial point, twice at trial midpoints, and once at a trial endpoint, as shown below.



Every trial evaluation uses the value of the function from the previous trial point, ie. k_2 is evaluated using k_1 and not using y_n. From these derivatives the final function value is calculated, The calculation is done using the equations,

$$\begin{aligned} k_1 &= hf(x_n, y_n) \\ k_2 &= hf(x_n + \tfrac{h}{2}, y_n + \tfrac{k_1}{2}) \\ k_3 &= hf(x_n + \tfrac{h}{2}, y_n + \tfrac{k_2}{2}) \\ k_4 &= hf(x_n + h, y_n + k_3) \end{aligned}$$

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{7.21}$$

The program **rk4-sine.py** is obtained by repacing function rk2 by rk4, as listed below.

```
def rk4(x, y, fxy, h):   # x, y , f(x,y)
    k1 = h * fxy(x, y)
    k2 = h * fxy(x + h/2.0, y+k1/2)
    k3 = h * fxy(x + h/2.0, y+k2/2)
    k4 = h * fxy(x + h, y+k3)
    return  y + ( k1/6 + k2/3 + k3/3 + k4/6 )
```

From the results 7.4(a) ,7.5(a) and 7.5(b) one can see the superiority of fourth order Runge-Kutta method. The program *compare-euler-RK4.py* evaluates sine function using both Euler and RK4 methods. The error is compared by plotting the differences with sin() function. It can be seen that the error is much less in the case of RK4

Example **compare-euler-RK4.py**

```
import numpy as np import matplotlib.pyplot as plt

def f1(x,y):
    return np.cos(x)

def euler(x, y, fxy, h):
    return y + h * fxy(x,y)   # Euler method

def rk4(x, y, fxy, h):   # x, y , f(x,y)
    k1 = h * fxy(x, y)
    k2 = h * fxy(x + h/2.0, y+k1/2)
    k3 = h * fxy(x + h/2.0, y+k2/2)
    k4 = h * fxy(x + h, y+k3)
    return  y + ( k1/6 + k2/3 + k3/3 + k4/6 )

dx = .1   # step size
xmin = 0  # initial value
xmax = np.pi # calculate up to this only
xa = np.arange(xmin, xmax, dx)
# array of the independent variable N =len(xa)
ya = np.empty(N) # numpy array to store results
sf = np.sin(xa) # sine function

ya[0] = 0.0 # given initial value
for i in range(N-1):
    ya[i+1] = euler(xa[i], ya[i], f1, dx)
plt.plot(xa, ya-sf, 'x')

for i in range(N-1):
    ya[i+1] = rk4(xa[i], ya[i], f1, dx)
plt.plot(xa, ya-sf, 'o')
plt.show()
```

7.5.4 Function depending on the integral

In the previous examples, the function depends on the independent variable only. Let us consider an example where it depends on y also. *The program rk4-xy.py* listed below evaluates the function

$f(x, y) = 2(y - x^2) + 1$. Readers may verify the results by manual computing.

Example **rk4-xy.py**

```
import numpy as np

def f1(x,y):
    return 2*(y-x**2) + 1   # derivative

def rk4(x, y, fxy, h):   # x, y , f(x,y)
    k1 = h * fxy(x, y)
    k2 = h * fxy(x + h/2.0, y+k1/2)
    k3 = h * fxy(x + h/2.0, y+k2/2)
    k4 = h * fxy(x + h, y+k3)
    ny = y + ( k1/6 + k2/3 + k3/3 + k4/6 )
    return ny

dx = 0.1   # stepsize
xa = np.arange(0,1,dx)
N = len(xa)
ya = np.empty(N)  # to store y values
ya[0] = 0.0    # initial value

for k in range(N-1):
   ya[k+1] = rk4(xa[k], ya[k], f1, dx)

for k in range(N):
   print (xa[k], ya[k],  xa[k]**2 + xa[k])
```

The results are shown below. The last column is the exact value, from $y = x^2 + x$. The program uses the derivative of this equation $\frac{dy}{dx} = 2x + 1$, where x is substituted with $(y - x^2)$ to get $2(y - x^2) + 1$.

0.0 0.0 0.0
0.1 0.109998333333 0.11
0.2 0.239996297667 0.24
0.3 0.389993811303 0.39
0.4 0.559990774459 0.56
0.5 0.749987065258 0.75
0.6 0.959982534839 0.96
0.7 1.18997700139 1.19
0.8 1.43997024283 1.44
0.9 1.70996198792 1.71

7.6 Second Order Ordinary Differential Equations

In order to solve a second order differential equation using numerical methods, we need to convert it in to two coupled first order equations, as shown below. The equation

$\frac{d^2y}{dx^2} = \frac{d}{dx}\left(\frac{dy}{dx}\right) = f(x, y)$

can be converted in to two coupled equations
$\frac{dy1}{dx} = f1(y1, y2, x) = y_2$ and $\frac{dy2}{dx} = f2(y1, y2, x)$
The program **second-ode-rk4.py** listed below integrates the equation $\frac{d^2y}{dx^2} = -y$,with the initial conditions $y = 0$ and $\frac{dy}{dx} = 2$ at $x = 0$. The variable x is passed on to the functions f1 and f2. even though they don't use it in this example. The result plotted shows that the solution is sine function, where the initial value of $\frac{dy}{dx}$ decides the amplitude.

```
import numpy as np
import matplotlib.pyplot as plt
dx = 0.1
xa = np.arange(0, 10, dx)   # array of x
N =len(xa)

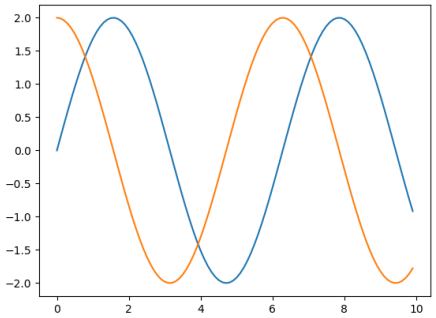
def f1(y1,y2,x):
    return y2   # derivative of y1 is y2

def f2(y1,y2,x):
    return -y1   # as per the equation

def rk4(y1,y2, f1, f2, x, h):
    k11 = h*f1(y1,y2,x);
    k21 = h*f2(y1,y2,x);
    k12 = h*f1(y1+0.5*k11,y2+0.5*k21,x+0.5*h);
    k22 = h*f2(y1+0.5*k11,y2+0.5*k21,x+0.5*h);
    k13 = h*f1(y1+0.5*k12,y2+0.5*k22,x+0.5*h);
    k23 = h*f2(y1+0.5*k12,y2+0.5*k22,x+0.5*h);
    k14 = h*f1(y1+k13,y2+k23,x+h);
    k24 = h*f2(y1+k13,y2+k23,x+h);
    y1 += (k11+2*k12+2*k13+k14)/6;
    y2 += (k21+2*k22+2*k23+k24)/6;
    return y1,y2

y1a = np.zeros(N)    # array to store the computed y1
y2a = np.zeros(N)    # and y2
y1a[0] = 0 # initial values
y2a[0] = 2 # of y1 and y2

for i in range(N-1):
    y1a[i+1], y2a[i+1] = rk4(y1a[i], y2a[i], f1, f2, xa[i], dx)
plt.plot(xa, y1a) plot(xa, y2a)
plt.show()
```



7.7 Solution of Algebraic Equations

In general, an equation may have any number of roots, or no roots at all. For example $f(x) = x^2$ has a single root whereas $f(x) = sin(x)$ has an infinite number of roots. The roots can be located visually, by looking at the intersections with the x-axis. Another useful tool for detecting and bracketing roots is the incremental search method. The basic idea behind the incremental search method is simple: if $f(x1)$ and $f(x2)$ have opposite signs, then there is at least one root in the interval $(x1, x2)$. If the interval is small enough, it is likely to contain a single root. Thus the zeroes of $f(x)$ can be detected by evaluating the function at intervals of Δx and looking for change in sign.

There are several potential problems with the incremental search method: It is possible to miss two closely spaced roots if the search increment Δx is larger than the spacing of the roots. Certain singularities of $f(x)$ can be mistaken for roots. For example, $f(x) = tan(x)$ changes sign at odd multiples of $\pi/2$, but these locations are not true zeroes as shown in figure 7.6 (b).

Program **rootsearch.py** listed below implements the function *root*() that searches the roots of a function $f(x)$ from $x = a$ to $x = b$, increasing it by dx.

```
def func(x):
   return x**3-10.0*x*x + 5

def root(f,a,b,dx):
    x = a
    while True:
       f1 = f(x)
       f2 = f(x+dx)
       if f1*f2 < 0:
         return x, x + dx
       x = x + dx
       if x >=  b:
           return (None,None)

x,y = root(func, 0.0, 1.0,.1)
print x,y
```

The program finds that the root is between .7 and .8. Let us explore other methods like Bisection method, Newton-Raphson method etc., to find the root with a specified accuracy.

```
x = np.linspace(0,10,NP)
data = 3 * x + 2 + r

xbar = np.mean(x)
ybar = np.mean(data)
b = sum(data*(x-xbar)) / sum(x*(x-xbar))
a = ybar - xbar * b
print a,b
y = a + b * x
plt.plot(x,y)
plt.plot(x,data,'ob')
plt.show()
```

The raw data is made by adding random numbers (between -1 and 1) to the y coordinates generated by $y = 3 * x + 2$. The Numpy functions mean() and sum() are used. The output is shown in figure 7.7.

7.11 Monte Carlo methods

We only provide a glimpse of this method using an example program to calculate the value of π. We consider a square shaped area ranging from the coordinates (0,0) to (1,1), and a quarter of a circle centered at (0,0). We randomly generate points in the range 0 to 1, representing the x and y coordinates of a random point inside the square. We calculate the distance to the point from the origin to find out whether it falls inside the circle. This probability is decided by the area of the circle (one fourth of it) and that of the square. The program **pi-value.py** is listed below. Accuracy of the result depends on the number of trials.

```
from random import random
from math import sqrt

NP = 100000
inside = 0
for k in range(NP):
    x = random()
    y = random()
    r = sqrt(x**2 + y**2)
    if r <= 1:
        inside += 1

ratio = inside / NP
print (4*ratio)
```

7.12 Fourier Series

A Fourier series is an expansion of a periodic function $f(x)$ in terms of an infinite sum of sines and cosines. The computation and study of Fourier series is known as harmonic analysis and is extremely useful as a way to break up an arbitrary periodic function into a set of simple terms that can be plugged in, solved individually, and then recombined to obtain the solution to the original problem or an approximation to it to whatever accuracy is desired or practical.

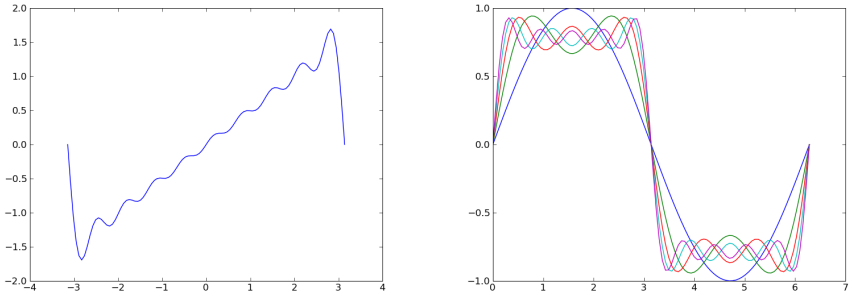


Figure 7.8: Sawtooth and Square waveforms generated using Fourier series.

The examples below shows how to generate a square wave and sawtooth wave using this technique. To make the output better, increase the number of terms by changing the argument of the range() function, used in the for loop. The output of the programs are shown in figure 7.8.

Example **fourier_square.py**

```
import numpy as np
import matplotlib.pyplot as plt
N = 100    # number of points
x = np.linspace(0.0, 2 * np.pi, N)
y = np.zeros(N)
for n in range(5):
    term = np.sin((2*n+1)*x) / (2*n+1)
    y = y + term
    plt.plot(x,y)
plt.show()
```

Example **fourier_sawtooth.py**

```
import numpy as np
import matplotlib.pyplot as plt
N = 100    # number of points
x = np.linspace(-np.pi, np.pi, N)
y = np.zeros(N)
for n in range(1,10):
    term = (-1)**(n+1) * np.sin(n*x) / n
    y = y + term
plt.plot(x,y)
plt.show()
```

7.13 Exercises

All problems are to be solved by writing Python code.

1. Evaluate $sin(\pi/8)$using Taylor series.

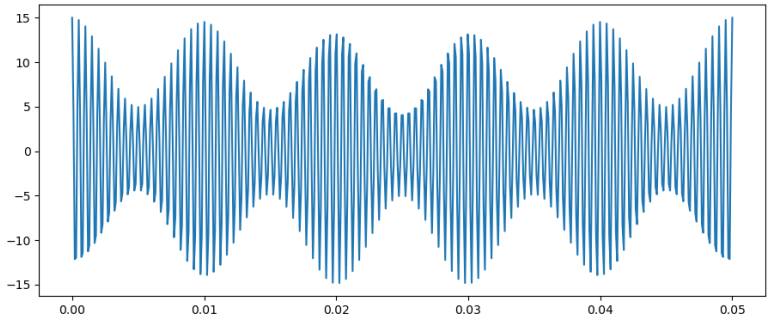


Figure 8.2: Amplitude Modulation

```
wave2 = np.sin(2*np.pi*f2*t)      # 104 Hz sine wave
plt.plot(t, wave1 + wave2)
plt.show()
```

8.2 Amplitude Modulation

$$s(t) = A_c \left[1 + \mu \cos(2\pi f_m t)\right] \cos(2\pi f_c t) \tag{8.1}$$

The eqation 8.1 represents amplitude modulation, where $s(t)$ is the modulated wave, A_c is the amplitude of the carrier wave, f_c the frequency of the carrier, $f_m$the frequency of the modulating signal and μ the modulation index. The program **am.py** listed below gives the output shown in the figure 8.2. The various input parameters, for example the modulation index, may be changed to view the results.

```
import numpy as np
import matplotlib.pyplot as plt
t = np.linspace(0, .05, 500)  # array of time
Ac = 10
mi = 0.5
Fc = 1.e6
Fm = 1.e2
res = Ac*(1+mi*np.cos(2*np.pi*Fm*t))*np.cos(2*np.pi*Fc*t)
plt.plot(t, res)
plt.show()
```

8.3 Radioactive decay

$$\frac{dN}{dt} = -\lambda N \tag{8.2}$$

The decay of an unstable nucleus is entirely random in time so it is impossible to predict when a particular atom will decay. But the number of decay events dN expected to occur in a small interval of time dt is proportional to the number of atoms present N, given by the equation8.2. Analytically solving it shows that the decay is exponential. We can also solve it by integrating

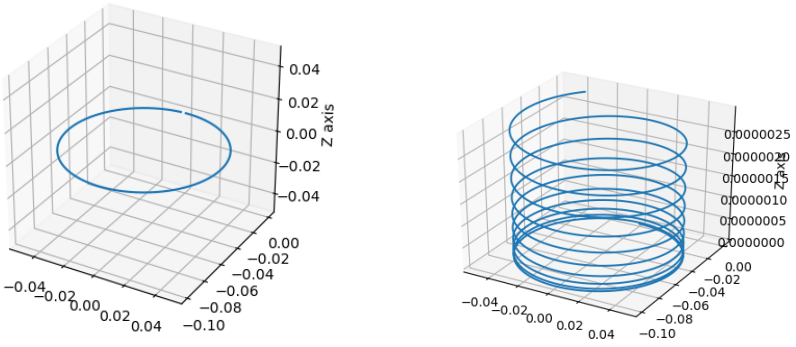


Figure 8.3: (a)Trajectory in magnetic field.(b) with E & M fields

the equation 8.2 numerically. The program **rdecay-scipy.py** listed below plots the number of radioactive atoms as a function of time.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import integrate
L = 5  # decay constant
def derivative(y, t0):
  return -L * y # dN/dt = -L * N
N = 1000 # value at t = 0
dt = 0.1
t = np.arange(0, 1, dt) #time span and steps
nt = integrate.odeint(derivative, N, t) # integrate
plt.plot(t, nt[:,0])  # extract the first column from the 2D array
plt.show()
```

8.4 Charged particle in E and M fields

Using the equations of Newtonian mechanics one can predict the position and velocity of a body if the initial position and velocity are given, The forces acting on it gives the acceleration. Integrating the acceleration gives velocity and position, enabling us to compute the trajectory of the body.

Computing the trajectory of a charged particle in electric and magnetic fields has numerous applications in science and technology, for example in the designing of particle accelerators. The values are taken from the first cyclotron accelerator made by E Lawrence `https://accelconf.web.cern.ch/Cyclotrons2010/talks/mom1cio02_talk.pdf`. We are calculating the trajectory of an 80 keV proton in a 0.82 Tesla magnetic field. The trajectory is traced for 79 nanoseconds, slightly less than one full circle. The radius of the orbit is around 10 cm. The program **lorentz-force-scipy.py** is listed below, and the output is shown in figure 8.3(a).

```
import numpy as np
from scipy import integrate
import matplotlib.pyplot as plt
```

The array being processed is passed as an argument. This array is not modified; the result of the calculation is returned in a new array.

If you need the extra performance and reduced memory usage that come from overwriting an array, use the information in Appendix D to plan your code carefully and avoid unintended consequences. However, try vectorizing your code using NumPy's efficient array operations before you resort to overwriting. Vectorized code will almost always run faster than loops you write yourself in Python.

5.2 RANDOM NUMBERS AND SIMULATION

There are many interesting problems in which we do not have complete knowledge of a system, but we do know the probabilities of the outcomes of simple events. For example, you know the probability of any given outcome in the roll of a single die is 1/6, but do you know how likely it is that the sum of a roll of 5 dice is less than 13? Rather than work out the combinatorics, you could roll 5 dice many times and determine the probability empirically.

A random number generator makes it possible for a computer to "roll dice" millions of times per second. Thus, you can use a random number generator to simulate a system described by a stochastic model in which the probability distributions of the parameters are known. You can determine the likely behavior of the system, even if you cannot you can't work out the details analytically. Such calculations are often called "Monte Carlo simulations."

5.2.1 Simulating coin flips

The simplest example of a stochastic system is a coin flip. Suppose that you want to simulate flipping a coin 100 times, record the number of heads or tails, and then repeat the series of 100 flips N times. This will generate a set of N numbers, each falling in the range of 0 to 100, inclusive. How do we get Python to flip the coin for us? First, try typing `1>2` at the IPython console prompt, and then `2>1`. You'll see that Python returns a Boolean value of **`True`** or **`False`** when it evaluates each proposition. You can simulate a coin flip by generating a uniformly distributed random number between 0 and 1, then checking to see if it is less than 0.5. If the comparison returns **`True`**, we record heads; if **`False`**, we record tails. Python can also use such values in numeric calculations: It converts **`True`** to 1 and **`False`** to 0.

The module **`np.random`** contains random number generators for several different probability distributions. The function **`np.random.random`** generates numbers from the "continuous uniform distribution" over the interval $[0, 1)$, and will serve our purposes for this chapter. You should explore the other functions in the **`np.random`** module as well. To save typing, we will import **`np.random.random`** under the nickname `rng`:

```
from numpy.random import random as rng
```

In the future, if you wish to switch to a different random number generator, this is the only line of code you will need to change: Just import a different function under the name `rng`.

To make a series of independent flips, first create an array called `samples` that contains 100 random numbers generated by `rng`. (Consult **`help`**`(rng)` for a clue about how to do this easily.) You can convert the random samples to a simulation of coin flips with `flips=(samples<0.5)`. Python applies the comparison item by item and stores the result in `flips`. You can then count the number

of heads by using `np.sum(flips)` or the array method `flips.sum()`. Repeat this several times to get a feel for how likely it is that exactly 50 heads occur in 100 flips of a fair coin.

5.2.2 Generating trajectories

We can adapt coin flipping to study random walks, Brownian motion, and a host of other interesting biological and physical systems. Let's create a random walk of 500 steps. Our trajectory will consist of 500 x values and 500 y values. Following good practice (Section 2.6.5), begin with

```
num_steps = 500
```

The idea behind our random walk is that steps along the x and y directions are statistically independent. You know from the preceding section how to get an array containing 500 random binary digits. Make two such arrays called `x_step` and `y_step`. For a coin flip, **`True`** and **`False`** or 1 and 0 were sufficient. For a random walk, we need a random string of $+1$ and -1 values.

Your Turn 5B

a. Do a simple operation on `x_step` to convert its entries to $+1$ or -1. Do the same for `y_step`.
b. Next, convert these arrays into successive positions of the random walker. Consult **`help(np.cumsum)`** to see why that function is useful.
c. Write a script to make a picture of your random walk, and run it several times.

Try writing a function called `get_trajectory` that will take the single argument `num_steps` and return the two arrays that contain the coordinates of a random walker.

5.3 HISTOGRAMS AND BAR GRAPHS

A **histogram** is a graphical representation of a discrete probability distribution. Suppose that you wish to check that the random number generator `rng` really gives a uniform distribution. To make a simple histogram, try

```
# histogram.py     [get code]
from numpy.random import random as rng
data = rng(100)
plt.hist(data)
```

A histogram appears, but a number of things have happened without your supervision. The function **`plt.hist`** has

- Inspected the list of data and determined its range;
- Divided that range into a number of equally spaced **bins**;
- Counted how many elements of `data` fall into each bin;
- Graphed the result as a set of bars; and
- Returned a tuple of two arrays and a strange-looking item called `<a list of 10 Patch objects>`.

to subdivide the integration range sufficiently, it will generally yield satisfactory results. The keyword *limit* controls how fine a grid **quad** is allowed to use. Try

```
quad(np.cos, 0, 5000)                # Results in a warning and enormous error!
quad(np.cos, 0, 5000, limit=1000)    # No warning; accurate result
np.sin(5000)                         # Exact result for comparison
```

5.7.4 T2 Parameter dependence

The **quad** function can only integrate functions of a single variable. If you have a function of several variables and you wish to integrate one of them, holding the others fixed, there are two options: You can define a dummy function of one variable and pass the dummy function to **quad**, or you can supply a keyword and value to **quad** that will specify the constant arguments. For example, suppose that you have defined a function as follows:

```
def f(x, a, b, c): return a*x**2 + b*x + c
```

To integrate the function for $(a, b, c) = (1, 2, 3)$, the two options work as follows:

```
# Use dummy function.
def g(x): return f(x, 1, 2, 3)
integral1, err = quad(g, -1, 1)
# Use keyword.
integral2, err = quad(f, -1, 1, args=(1, 2, 3))
```

If the variable you wish to integrate is not the *first* argument of the function, you cannot use the second method. You must define a dummy function.

5.8 NUMERICAL SOLUTION OF DIFFERENTIAL EQUATIONS

In the physical and life sciences, it is often possible to write down a set of differential equations that govern a system or describe the behavior of a model. However, it may be impossible to solve this system in terms of known functions. A classic example is the three-body problem in classical mechanics: $\boldsymbol{F} = m\boldsymbol{a}$ and Newton's law of gravitation are all that is required to write down the differential equations, but no one in the last four centuries has been able to find a general solution!

Numerical solution is a powerful tool in studying such systems.[8] Starting from some initial configuration of a system, you can calculate the next configuration from the differential equations. From *that* configuration, you can calculate the *next* configuration using the differential equations, and so on. Computers are ideal for executing this repetitive operation, and efficient libraries have been developed for the task.

To solve differential equations, we turn once again to SciPy. The module we need is called **scipy.integrate**, the same module that contains **quad**. There are several functions available in the module, but we will restrict our attention to the one called **odeint**:[9]

```
from scipy.integrate import odeint
```

[8] Many authors use the phrase "numerical integration of an ODE" to denote the procedure we call "numerical solution of an ODE." We will reserve the word "integration" for actually performing integrals, a procedure that sometimes, but not always, can be used to solve ODEs.

[9] T2 This function is adapted from the highly optimized ODEPACK library of FORTRAN functions. Calling **odeint** invokes a routine called LSODA, an adaptive solver that chooses a predictor-corrector method or a backward differentiation formula depending on its progress with the problem.

5.8.1 Reformulating the problem

An ordinary differential equation (ODE) is one that describes a function of a single independent variable, which we'll call $y(t)$. A textbook example is the driven harmonic oscillator:

$$\frac{\mathrm{d}^2 y}{\mathrm{d}t^2} = -y + g(t).$$

The **odeint** function can "only" solve ODEs of the form

$$\frac{\mathrm{d}\boldsymbol{y}}{\mathrm{d}t} = \boldsymbol{F}(\boldsymbol{y}, t). \tag{5.1}$$

Fortunately, *any* explicit ODE can be put into this form. For example, to describe the second-order ODE for the driven harmonic oscillator to **odeint**, we must reformulate it as a system of coupled first-order equations. First, define two new variables that will be the components of the vector $\boldsymbol{y}$:

$$y_1 = y \qquad\qquad y_2 = \frac{\mathrm{d}y_1}{\mathrm{d}t}.$$

Next, write the derivatives of y_1 and y_2 in terms of y_1, y_2, and t:

$$\frac{\mathrm{d}y_1}{\mathrm{d}t} = \frac{\mathrm{d}y}{\mathrm{d}t} = y_2 \qquad\qquad \frac{\mathrm{d}y_2}{\mathrm{d}t} = \frac{\mathrm{d}^2 y}{\mathrm{d}t^2} = -y + g = -y_1 + g.$$

This allows us to cast our problem, a second-order differential equation, in the form required by **odeint** (Equation 5.1). Here, $\boldsymbol{y}$ is an array with two entries, and $\boldsymbol{F}(\boldsymbol{y}, t)$ returns an array with two entries:

$$\boldsymbol{F}(\boldsymbol{y}, t) = \begin{bmatrix} y_2 \\ -y_1 + g \end{bmatrix}. \tag{5.2}$$

Any ODE can be recast in a similar manner.

5.8.2 Solving an ODE

You must supply three arguments to **odeint**: the function $\boldsymbol{F}(\boldsymbol{y}, t)$, an array that defines the initial conditions, and an array of values at which you'd like **odeint** to evaluate the solution $\boldsymbol{y}(t)$. The general function call is

```
y = odeint(F, y0, t)
```

The variables in the expression are

F – A function $\boldsymbol{F}(\boldsymbol{y}, t)$ that accepts an array and a scalar and returns an array.

y0 – A one-dimensional array with the initial values of $\boldsymbol{y}$.

t – An array of t values at which $\boldsymbol{y}$ is to be computed. The first entry of this array is the time at which the initial values `y0` apply.

y – An array of the values of $\boldsymbol{y}(t)$ at the points specified in `t`.

To determine the motion of an undriven harmonic oscillator (the example above with $g(t) = 0$), we first need to define the function $\boldsymbol{F}(\boldsymbol{y}, t)$. The first argument, `y`, is an array that contains the values of y_1 and y_2 at time t. The function we need is

```
# simple_oscillator.py      [get code]
def F(y, t):
    """
    Return derivatives for second-order ODE y'' = -y.
    """
    dy = [0, 0]          # Create a list to store derivatives.
    dy[0] = y[1]         # Store first derivative of y(t).
    dy[1] = -y[0]        # Store second derivative of y(t).
    return dy
```

Note that `F(y,t)` requires time as an argument even though the value of `t` is not used in this example. You will get an error if you omit this argument.

We can now study the motion of the harmonic oscillator for different initial conditions:

```
# solve_ode.py     [get code]
""" ODE solver for harmonic oscillator. """

import numpy as np, matplotlib.pyplot as plt
from scipy.integrate import odeint

# Import ODE to integrate:
from simple_oscillator import F

# Create array of time values to study:
t_min = 0; t_max = 10; dt = 0.1
t = np.arange(t_min, t_max+dt, dt)

# Provide two sets of initial conditions:
initial_conditions = [ (1.0, 0.0), (0.0, 1.0) ]

plt.figure()     # Create figure; add plots later.
for y0 in initial_conditions:
    y = odeint(F, y0, t)
    plt.plot(t, y[:, 0], linewidth=2)

skip = 5
t_test = t[::skip]                          # Compare at a subset of points.
plt.plot(t_test, np.cos(t_test), 'bo')      # Exact solution for y0 = (1,0)
plt.plot(t_test, np.sin(t_test), 'go')      # Exact solution for y0 = (0,1)
```

This code illustrates the following:

- Like **quad**, **odeint** expects its first argument to be a function name (with no parentheses).
- The second argument is a vector with two entries, specifying the initial conditions.
- The return value of **odeint** is an array. The first column is $y(t)$ evaluated at the values in `t`; the second is $\mathrm{d}y/\mathrm{d}t$. If you solve a higher order ODE, the third column will contain $\mathrm{d}^2y/\mathrm{d}t^2$, and so on.

Your Turn 5G

Modify `simple_oscillator.py` to incorporate the driving force $g(t) = \sin(0.8t)$, and comment on how the solution changes.

5.8.3 T2 Parameter dependence

If we wanted to explore several different harmonic oscillators, we might consider modifying the function `F(y,t)` as follows:

```
# parametric_oscillator.py      [get code]
def F(y, t, spring_constant=1.0, mass=1.0):
    """
    Return derivatives for harmonic oscillator:
        y'' = -(k/m) * y
    y = displacement in [m]
    k = spring_constant in [N/m]
    m = mass in [kg]
    """
    dy = [0, 0]          # Array to store derivatives
    dy[0] = y[1]
    dy[1] = -(spring_constant / mass) * y[0]
    return dy
```

To specify fixed parameters when using **odeint**, there are two options, just as for **quad** in Section 5.7.4. First, we can define a dummy function that sets the parameters and pass this function to **odeint**. Alternatively, we can supply an optional argument to **odeint** by using the keyword *args*. Suppose that the system has a spring constant of 2.0 N/m and a mass of 0.5 kg.

```
y0 = (1.0, 0.0)
t = np.linspace(0, 10, 101)

# Use dummy function.
def G(y, t): return F(y, t, 2.0, 0.5)
yA = odeint(G, y0, t)

# Use keywords.
yB = odeint(F, y0, t, args=(2.0, 0.5))
```

To use the second method, the parameters you wish to set must come after `y` and `t` in the list of arguments to `F`. Otherwise, you must define a dummy function.

5.9 VECTOR FIELDS AND STREAMLINES

A vector field is a function whose value at any point in space is a vector. Common examples from physics include the electric field, the gravitational field, and the velocity field of a fluid. PyPlot provides two useful functions for visualizing vector fields and their streamlines: **plt.quiver** and **plt.streamplot**.